Algorithmic Trading Using Python - Full Course

(0:00 - 0:07)

Hello, and welcome to this free CodeCamp course on algorithmic trading in Python. My name is Nick McCollum. I'm going to be your instructor for this course.

(0:08 - 0:26)

And in this course, we're going to focus on building three large quantitative finance projects. In the first project, we are going to build an equal weight version of the popular S&P 500 index fund. In the second project, we are going to build a quantitative momentum strategy that selects the best stocks based on a variety of momentum investing metrics.

(0:26 - 0:45)

And in the third project, we're going to build a quantitative value screener that selects stocks that are attractive based on a number of value metrics. So that's a quick summary of the three projects that we are going to build in this course. Before we move into learning more about the course and those projects, I have a couple of housekeeping items that I have to go over first.

(0:46 - 1:02)

To start, this free CodeCamp course on algorithmic trading in Python is made possible through a grant from IEX Cloud. If you've ever read the book Flash Boys by Michael Lewis, you may have heard of IEX. In this course, we're going to be using some of their tools and APIs to populate some of the data that we will need for these algorithmic trading strategies.

(1:03 - 1:22)

The other quick thing that you're going to notice starting now is a little message at the bottom of the screen that's going to stay there for the rest of the video. And what that message intends to represent is that this course is for educational purposes only. I'm going to be providing some high level finance concepts, but nothing in this course or in this video should be construed as investment advice.

(1:22 - 1:36)

So just to reiterate, this course is for educational purposes only, and none of this should be considered investment advice. So with all of that out of the way, let's dig into a little bit more about what this course is going to teach you. So let's start by just going over a quick course overview.

(1:37 - 1:54)

The first thing that we're going to discuss is some basics of the field of algorithmic trading.

Then we're going to discuss some API basics and kind of just quickly go over how the course is configured and laid out. The next thing we're going to discuss is the first project of this course, which is an equal weight version of the popular S&P 500 index fund.

(1:55 - 2:49)

And then we're going to talk about our second project, which, as I mentioned earlier, is a quantitative momentum strategy. And then we're going to discuss briefly our third project, which is a quantitative value strategy. So that's kind of a 30,000 foot view of what you're going to be learning in this course.

Let's zoom in a little bit and talk about some algorithmic trading basics. So for anyone who's completely unfamiliar with algorithmic trading, it basically means using computers to make investment decisions. So in the past where you may have had a team of financial analysts or investment researchers to kind of consider stocks and decide what to buy, many of the popular investing strategies that are implemented today actually use more computers than they do humans.

So algorithmic trading just means using computers to make investment decisions. Now, there are many different types of algorithmic trading. On the one hand, you have, you know, your super high frequency trading, big brain, really, really sophisticated and complex strategies.

(2:49 - 3:17)

And then at the other end of the spectrum, you can kind of basically map any type of fundamental strategy that humans have implemented in the past and just make it a little more efficient by providing some computer software in there. So as, you know, this slide kind of alludes to the main difference between the many different types of algorithmic trading is generally the speed at which those trades are executed. Now, before we actually dig into more about the world of algorithmic trading, it's kind of helpful to have a high level understanding of who the main players are in this space.

(3:18 - 3:35)

And one of the reasons I wanted to kind of provide this information at the start of this course is so that you can see that the feel of algorithmic trading is very big. And if you do want to make a career of it, there are lots of employers, lots of opportunities and lots of jobs in this space. So right now, I'm just going to briefly discuss a few of the largest players in the algorithmic trading landscape.

(3:36 - 3:56)

The first one, which is kind of the 50,000 pound gorilla in the room is Renaissance Technologies. I believe they're based in Long Island and they have 165 billion dollars in assets under management. Now, Renaissance Technologies is mostly famous for their Medallion Fund, which

is only open to employees at this point, which I think has returned something insane, like 50% a year for 20 years or something like that.

(3:56 - 4:13)

Now, none of us can access that fund because it's for employees only. But Renaissance Technologies probably has one of the best performing investment funds in the history of finance. And perhaps most interestingly is that Renaissance Technologies was not founded by a finance major or a investment analyst to start.

(4:13 - 4:26)

The person who actually started Renaissance Technologies was a math PhD from, I believe, a California university. His name's Jim Simons and many people would kind of consider him to be one of the forefathers of quantitative finance. So that's kind of the largest player in this space.

(4:27 - 4:43)

Another popular one is AQR Capital Management, which was founded by Cliff Assis and a few other managing partners. And they have 61 billion dollars in assets under management. Now, unlike Renaissance Technologies, AQR Capital Management actually has strategies that you and I can invest in through mutual funds and other types of strategies.

(4:43 - 5:00)

And one last interesting thing about that firm is that their name is kind of the IBM of investment firms, where IBM is International Business Machines, a very bland name, AQR Capital Management. The AQR component of that name actually stands for Applied Quantitative Research. So if you're ever curious what that stands for, that's what it is.

(5:01 - 5:24)

The last one is Citadel Securities, which has 32 billion dollars in assets under management, and they're a much more high frequency trading firm than the first two that I mentioned, kind of famous for their role in the market making space. So they are founded by Ken Griffin, and that kind of summarizes three of the large players. You can tell by just the assets under management of these three firms alone, more than 200 billion, that there are lots of opportunities and jobs, and this is a very large space.

(5:24 - 5:40)

So definitely something worth learning a bit about. Now, in this course, as you probably can tell by the title, we are going to be using Python for these algorithmic trading strategies. Now, before we dig into actually how to use Python and how to write the code for it, there's a few high level things you should understand.

(5:41 - 5:57)

The first one is that Python is probably the most popular programming language for algorithmic trading, and the reason why is just because it has a lot of libraries. But the downside to using Python is that it's a fairly slow programming language. So if you want to execute high performance code, Python is typically not your best bet.

(5:58 - 6:27)

Now, the solution that many practitioners have found for this is that Python is often used as a glue language to trigger code that actually runs in other languages. A common example of this is the NumPy library for Python, and that's actually a library that we'll be using in this course. Now, NumPy is the most popular Python library for performing numerical computing, and it's perhaps most popular for its data structure called the NumPy array, which easily allows you to store and manipulate one or two dimensional data structures in Python.

(6:27 - 6:59)

Now, although NumPy is actually a Python library, and it is called and manipulated using the Python programming language, the core underlying functionality is written in C, which is a much faster language and provides lots of performance enhancements to Python users who use the NumPy library in their code. So just to quickly summarize, the main idea of this section is that Python is the most popular programming language for quantitative finance, but it's also a slow language. So a lot of times you will use Python to trigger functionality that actually runs in other programming languages that are faster.

(7:00 - 7:11)

So now let's quickly talk about the algorithmic training process can kind of be broken down into the following steps. The first step is to collect data. The second step is to develop a hypothesis for a strategy.

(7:11 - 8:40)

The third step is to back test that strategy. Now, back testing just means formulating your strategy and then seeing how it would have performed historically over time. And generally, there's kind of two ways you want to do that.

You want to take the strategy back as far back in time as you can, and across as many markets as you can. So to provide a quick example, let's say that you have a hypothesis that the largest firms outperform, you would probably want to test that here in the United States and go back as far back in time as you can. And then you would also want to test the performance of the largest firms in Canada, Europe, Japan, China, India, all the other international markets as well.

And if that strategy performs well in all of those markets, then you can be sure that you're

probably onto something. Now, the fourth strategy would be to implement that or sorry, the fourth step rather would be to implement that strategy in production. So you've collected data, you develop a hypothesis, you back tested the strategy, and now you're going to start actually trading that strategy with real money in real accounts, moving forward and seeing how it does with real money in the strategy.

Now that you have a high level understanding of the algorithmic trading process, there's a few ways that this course will be different from a real algorithmic trading strategy. And I just want to highlight those quickly right now before we dive into some information about the three projects we'll be building. Now, just to quickly summarize, there's three major ways that this course will be different.

The first is that we'll be using random data. The data provider that we're using for this course, which is IES Cloud, is a paid API. So if you want to use real data, you can follow the steps in this course, but actually pay for an account and use a real API token.

(8:42 - 16:25)

What we'll be using instead is we will be using a sandbox API token, which provides random data, but it's also free. So what that means is you can, you can do all the normal things that you can with the IES Cloud API, but the data will be randomly generated. So you may see some interesting results because of that.

The second thing that is different is that we will not actually be executing trades in this course. Now, the reason why that is, is because many of the people who take this course will be using different custodians to execute their trades. And each one of those custodians will have a different API infrastructure.

So I wanted to make this course as widely applicable as possible. So what we're going to be doing instead is we're going to be imagining that we work at a firm where our traders expect us to generate a sell documents and send them to them to execute trades. So the output of each of these three strategies will be an Excel document that has the name of each companies that need to be purchased and the number of shares of each one of those companies that the trader needs to purchase.

So we won't be executing trades, but instead we will be generating order sheets to send to our imaginary traders further down the line. And just to reiterate that second point, we're going to be saving these recommended trades to Excel files, which could then be sent to the traders afterwards. All right.

So now that we have a bit of a high level understanding of the algorithmic trading landscape, it's time to talk about API basics and course configuration. So to start, let's discuss what is an API? API is an application programming interface. And you probably aren't familiar with that if you've ever heard the term API before, but that doesn't actually tell you what it does.

I think the easiest way to understand what an API is, is it's a way for your software to interact with and potentially control someone else's software. So in this case, we're going to be using the IEX Cloud API. And what we're going to do is use their API to access their database of financial data and import it into our Python script.

So like I say, APIs allow you to interact with someone else's software using your own code. And in this course, we're going to be using the IEX Cloud API to gather stock market data to make investment decisions. And to provide you a quick example of what that looks like, here's a code snippet.

So this is a cell from a Jupyter notebook that has four lines. You can see on the first line that we declare a symbol AAPL, and that is the stock symbol for AAPL on the NASDAQ exchange. On the second line, we're creating a string called API URL that has some interpolated values that interpolate and the symbol and our IEX Cloud API token.

And on the third line, we're using the requests library of Python to provide a get request to that API URL. And then on the last time, we're just printing the data that gets returned. Now, this is kind of a lot.

And if you don't fully understand this, not to worry, you'll see this in a lot more detail when we actually build the projects for this course. So that's kind of what an API call looks like. And broadly speaking, in this course, we're only going to be using get requests to gather data from the IEX Cloud API.

Now, if you've never used an API before, there's many different ways you can interact with them, but there's four major ones. And I'm going to show you briefly some of the other ways to interact with an API right now before we proceed through this course. So get requests allow you to get data from the API, and that's what we'll be using in this course.

An alternative is a post request, and that is a method for adding data to the database that's exposed by the API. So a get request allows you to get data from the API and a post request allows you to push data to the API. A put request allows you to add and overwrite data in the database that's exposed by the API.

Now you can think of a post request as a create only request. And you can think of a put request as a create or replace request. So there's a slight difference there.

And but even though it's a small difference, it's an important difference because that's kind of a common source of bugs for software developers. And the fourth kind of important type of API call is a delete, which as it may implies, it deletes data from the API's database. So to provide a quick summary of this section, we're going to be using exclusively get requests in our course, but there's three other main types of requests that you should understand those are post, put and delete.

Now, like most software concepts, APIs are best learned through rigorous practice. Because of

that, here is a URL that you can use to get a long list of public API's that are great practice for you to interact with API's. Now, I would probably recommend if you've never used an API before to proceed through this course, that will give you a base fundamental understanding of how to work with API's.

And then once that course is done, you can go to this public URL and practice with some other API's that are of interest to you personally. So this is not something for you to look at right now, but it's a good resource for you to hit later on. Alright, so now that you have a high level understanding of the course configuration and some API basics, let's move on to talking about our equal weight S&P 500 fund.

So if you've never heard of it before, the S&P 500 is the world's most popular stock market index. And many investment funds are benchmarked to the S&P 500. That means that they seek to replicate the performance of this index by owning all of the stocks that are held in the index.

Now, if you've never really researched how stocks are added to the S&P 500, there's a bit of a, there's a committee process and some, some sophistication there. But the easiest way to understand it is that the S&P 500 broadly covers the 500 largest companies in the United States. So if you own the S&P 500 index fund, you basically own the 500 largest companies in the US.

Now, aside from the fact that it owns the 500 largest companies, one of the other most important characteristics of the S&P 500 is that it is market capitalization weighted. Now, market capitalization is just a finance term that means basically size. So coming with a large market capitalization just means that it's a large company.

And what this means is that larger companies get a correspondingly larger weight in the S&P 500 index. For the first project of this course, we're going to build an alternative version of the S&P 500 index fund that provides that, that assigns the same weight to each company in the index. So instead of Apple having a very big way and Best Buy having a small way, every company will have the same weight.

So that's going to be the first project of this course. In the second project, we are going to build a quantitative momentum screener. Now, momentum investing means investing in assets that have increased in price the most.

To help you understand, let's kind of go through a quick example. Imagine that you have the choice between investing in two stocks that have had the following returns over the last year. Apple has stock has gone up 35% and Microsoft stock has gone up 20%.

A momentum investing strategy would suggest investing in Apple because of its higher recent price return. There are many other nuances to momentum strategies, including the concept of high quality momentum. And that basically means all else being equally good we want a stock that has steadily increasing price rather than no price change for a while and then a large jump at the end.

But we'll explore those when we actually build our quantitative momentum strategy later in this course. The third project we're going to build is a quantitative value screener. Now, value investing just means investing in stocks that are trading below their perceived intrinsic value.

So if you kind of want to make a very simple analogy, value investing is kind of the idea of buying a dollar for 75 cents and hoping that you can sell it again for a dollar later. So value investing is a very popular investing strategy because many of the world's best historical investors like Warren Buffett, Seth Clareman, and Benjamin Graham have employed this strategy. Creating algorithmic trading investing strategies relies on a concept called multiples.

And multiples are simply a way that investors use to estimate how valuable a company is. So to be more specific, multiples are calculated by dividing a company's stock price by some measures of the company's worth, like earnings or assets. So three examples of multiples that are used in value investing are the price to earnings ratio, the price to book ratio, and the price to free cash flow ratio.

(16:25 - 16:43)

Now, as their name applies, the price to earnings ratio is calculated by dividing a company's stock price by its earnings per share. The price to book value ratio is calculated by dividing a company's stock price by its book value per share. And the price to free cash flow ratio is calculated by dividing a company's stock price by its free cash flow per share.

(16:43 - 17:21)

So lots of different ways you can calculate multiples, but those are kind of a three, three of the most important ones anyway. Now, each of the individual multiples used by value investors has its pro and cons. One way to minimize the impact of any specific multiple is by using what's called a composite.

Now, a composite is just a average of many different valuation strategies. So in our strategy that we build in this course, we're going to actually be using a composite of five different value investing metrics. Now, with all that out of the way, you have a solid understanding of what we're going to be doing in this course.

And it's time for us to dig into the first project. So without further ado, let's do just that. All right.

(17:21 - 17:59)

So it's now time for us to start tackling our first project. If you skipped ahead to this section of this course, there's a couple of things that you should do before proceeding. So the first is to rewind back to the last section and install all of your dependencies.

If you're a pretty experienced Python developer, the main dependencies for this course are

NumPy, Pandas, and XLXX writer. So if you already have those on your machine, that's fine. If you've already been following throughout this video so far, I'm in our algorithmic trading Python folder that we cloned to our local computer.

I have my virtual environment activated. And what I'm going to do now is launch the Jupyter notebook for our first project. So to do that, the command is python dash M and then notebook.

(18:00 - 18:15)

So this should open up a browser window. And what we want to do is navigate into the starter files folder of the algorithmic trading repository and then open up the equal weight SMP 500.iPyMb notebook. It should look like this.

(18:15 - 18:25)

So basically, this is what we're going to be working through in this video. It has some instructions written in markdown and some blank code cells. And that's what we're going to complete as we work through this section.

(18:25 - 18:43)

So let's start with just some background information. What we're going to do in this project is to build an equal weight version of the SMP 500 index fund. The SMP 500 is an equal weight or not equal weight market weight market cap weighted basket of the 500 largest companies in the United States.

(18:43 - 20:39)

Interestingly enough, it actually has more than 500 stocks in it because five or so of the largest companies in the U.S. actually have multiple share classes. So an example of that is Google. It has a class A and a class C share that trade on the stock exchange.

So since Alphabet, the company is included in the SMP 500, both of its stocks are. And because of that, there's usually around 505 stocks in the SMP 500, despite the index only representing 500 companies. So anyways, to move on, what we're going to do now is create a different version of the SMP 500 index fund that doesn't weight the stocks in the index by market capitalization, but instead equal weights them.

So what this means is that the larger companies in the index will have less weight than the traditional version. And the smaller companies in the index will have a larger weight than the traditional version. So there's some background information here.

It just says that the largest fund that's benchmarked to this index is the SPDR SMP 500 ETF trust. That's commonly known as the ticker SPY SPY, which is the ticker that it trades on on the stock exchange. And that ETF has more than $250 billion of assets under management, which is

pretty crazy.

Most asset management firms don't have that much AUM. So a really big fund, to be sure. And it says here, the goal of this section of the course is to create a Python script that will accept the value of your portfolio and tell you how many shares of each SMP 500 constituent that you need to purchase to get an equal weight version of the index fund.

So the first thing we need to do is, you know, like most Python scripts, this project will rely on a number of open source software libraries. So we're going to import those first, we install them to our local computer in the dependency section of this course. So now we just need to import them and we're going to go through them one by one here.

And I'll briefly explain what each of these libraries does in case you haven't worked with them in the past. Alright, so the first library we're going to import is NumPy. And the command to do that is import NumPy, NumPy as NP.

(20:39 - 22:08)

Now, NumPy is a numerical computing library and it's known for its fast execution speed. And the reason why NumPy is so fast is because it's actually a C or maybe C plus plus module. So when you call a Python function in NumPy, it actually executes the code in a different programming language and that programming language is kind of faster by design.

So NumPy is often used in finance and other applications to speed up, you know, basic functions like summing or multiplication because it has modules written in a different programming language that is naturally faster. So that's NumPy. We're also going to import pandas as PD and pandas is a portmanteau for panel data or panel data is, I guess, because it has an S on it.

And what pandas does is it makes it very easy to work with tabular data in Python. So tabular data is anything that has rows and columns. You can think of an Excel spreadsheet as perhaps one of the most commonly known versions of tabular data.

And pandas is most widely used for its data structure called the pandas data frame and a data frame is just a data structure that holds tabular data. So pandas allows you to store data in a data frame and then use many different built in pandas functions and methods to manipulate the data within that data frame. You'll see lots of examples about this as you work through the rest of this course.

And if you're interested in learning more about pandas, free code camp has an excellent YouTube video on the topic that I would highly recommend. So that's pandas. And then there's a couple of others we need as well.

(22:09 - 25:31)

So actually, before we move on, one thing that's worth talking about is these aliases. So we didn't just import NumPy, we imported NumPy as MP and we didn't just import pandas, but we imported pandas as PD. So why do we do that? This just saves us a bit of typing because we're going to be often calling functions from these libraries.

So an example would be let's just run this so those imports run and then create a new code cell. If we wanted to create a pandas data frame that, you know, we'll just create an empty one as for an example. If we had just imported pandas without importing it as PD, we would have to write pandas.dataframe. And this calls the class that instantiates a pandas data frame object.

Now, because we imported it under the alias PD, we can just do PD.dataframe and it's a little bit more readable and faster to type. So it's not necessary to import them under aliases. I know a few developers who just run pandas.dataframe, but importing pandas under the alias PD and NumPy under the alias NP is pretty common and kind of widely considered to be a best practice.

So that's what we're going to follow in this course. All right, let's move on to importing the other libraries we need. So the next one is requests.

And requests is a very popular Python library that is kind of considered the gold standard for making HTTP requests. Now, an HTTP request is just basically an internet request that you can send to an API to get back some data. And in this course, we're going to be using the requests library to execute our API calls to the IEX cloud API to, I guess, receive our stock market data that we need to calculate the weightings for our S&P 500 index funds.

So the request library is for HTTP requests. And that's basically all you need to know about it for now. And the next thing we need to import is XLSX Writer.

This is a library that makes it very easy to save well formatted documents, Excel documents from a Python script. So that's XLSX Writer. And the last library we need is Math.

So Math is just a basic Python library that provides many of the basic mathematical functions that you need to, I guess, execute operations within a Python script. So all right. So once you run that code, all of your libraries will be imported.

And the next thing that we need to do is to import our list of stocks. Now, like I said, this is going to be a list of about 500 stocks, maybe a little more if there's any stocks that are dual listed in the S&P 500. And one thing to note about this project is that since these constituents change over time, in an ideal world, you would actually connect right to an index provider or an API, a financial data API or something like that, so that if a stock is added or removed from the S&P 500, then your equal weight S&P 500 strategy would reflect that.

Now, all of the tools that would provide that information are paid. And this is a free course. So I didn't want to provide any paid resources in the free course that kind of defeats the point of it being free.

So what what I did instead was I saved a list of the 500 stocks in the S&P 500 into a CSV file. And you can click this link here to download that file. So what we need to do is click that it'll go into our downloads folder and then we need to move that file into our starter files, starter files, excuse me, we need to move this file into our starter files folder so that it can be accessed by the other files in that directory.

(25:31 - 26:19)

So let's do that now. And then once that's done, we can import it into our Jupyter Notebook. So open up your finder app or I guess it would be finder app or Windows Explorer or whatever app you have on your operating system to explore files and navigate to your downloads folder in one tab.

And then in your other tab, you're going to want to navigate into that algorithm we're trading in Python course. So here it is. And then specifically, you want to navigate into the starter files folder and then drag this S&P 500 stocks dot CSV into the starter files folder like that.

Okay, so now that that's done, what we need to do is save the S&P 500 stocks as a pandas data frame. And the way to do that is we're going to assign it to a pandas data frame named stocks. So stocks equals P dot read CSV, and then the name of the file.

(26:19 - 26:26)

So if you just type in SP and then tab, it might auto complete. There it is. And if it doesn't auto complete, just type in the file name.

(26:26 - 29:08)

So what is this? It is a, this is a method that lives within the pandas library that takes in data in the form of a CSV and then stores it in the pandas data frame. So you can verify that by just doing type of stocks and this should return pandas data frame. Yep.

And then if you print it out, it'll actually show the data. So Jupyter notebooks have this kind of special feature where if you type a variable on the last line of a code seller, we'll print it. So this is what is contained in our, in our pandas data frame.

Like I said, there's actually 505 stocks within this data frame, which means that there's five stocks that have dual share share structures in the S&P 500 at this time. So that's what we're going to be working with throughout the rest of this tutorial. The next thing we need to do is acquire an API token.

Like I mentioned in the introduction to this course, we're going to be using the IEX cloud API to gather all of our financial data. And like most APIs, it requires some form of authentication before you can pull data from it. So what we're actually going to be using in this course is the sandbox mode of the IEX cloud API.

And sandbox mode is a kind of like a play mode where you can make sure that all of your API calls work, but it doesn't actually return real time financial data. Instead, it returns randomized financial data. And the reason why sandbox mode exists is because it allows you to test whether or not your API calls function properly before you actually start to use the real API and incur data usage as a result.

So the easiest way to handle this, in my opinion, when building this course was to create a secrets.py file. And secrets.py files are files that are stored in your repository or on the server that's running this code that doesn't actually get pushed to GitHub or any kind of remote git store. So the reason why is because most of the information stored in a secrets.py file is sensitive or confidential in some way.

And you just don't want that type of information stored in a centralized git repository. So you'll notice in the git ignore of this course, the secrets.py file is actually included in the git ignore, which means that if you ever fork this repository to work on it and then push your changes up to GitHub, the information contained in the secrets.py file doesn't actually get pushed up as well. So since it's actually a sandbox API key, it doesn't really matter in this specific instance, whether the secrets.py file is exposed.

But keep in mind for future Python courses or anything else that you work on that, you should never share your secrets.py file with anyone. But this course is kind of an exception since we're working with a sandbox API key. So to start, we're going to click here and this will download a secrets.py file to your downloads folder.

(29:08 - 33:58)

Now you may get a warning that says something like this. In my case, it says this type of file can harm your computer. Do you want to keep secrets.py anyway? And we're going to keep it and then move it into the same starter files folder that we moved our CSV file of stocks into.

So if you open back up your finder app or whatever app that you have on your operating system, you should have your two tabs still open. You can click and drag the secrets.py file into the starter files folder. And then we need to import our IEX cloud API key into our script.

So what we're going to do for that is we're going to say from secrets, import IEX and then if you hit tab, it should auto complete. It says cannot import IEX cloud API token from secrets. I wonder what that is.

User download. Okay, so this is equal is 100. All right.

So sometimes what happens with these Jupyter notebooks is that when you add a file to the working directory that the Jupyter notebook is open in, it actually doesn't recognize that it's there until you reset the kernel. So let's try that. We're going to say, restart the kernel, and then we're going to go back to the top and run all of our code cells from the start.

Awesome. And this time it imported correctly. So in a lot of cases, I might run into little hiccups like that when working through this code.

And in some courses, people edited out for a more clean experience. But I would say in the majority of cases, I'm going to leave them in so that you can see the debugging process and what I actually do to fix the problems that I encounter. So that was a small example of that.

You'll see more as you work through this course. So now we're going to make our first API call. So making API calls is a bit of an art because every API works a little bit differently.

And some APIs have lackluster documentation. In this case, the IEX cloud API has excellent documentation. And I'm going to kind of show you a bit of the documentation as we work through this section that creates our first API call.

So in this specific case for the S&P 500 index fund, there's two things we need. We need the market capitalization for each stock and the price of each stock. So to start, we're just going to do an API call for one single stock.

And then to generalize that for our entire universe of stocks, what we're going to do is loop over every stock that's contained in the pandas data frame that we created earlier and then run that same API call it for each one of those stocks using a Python for loop. So to start, let's just create symbol equals, and we're going to make this a string that is apples toxic, which is AAPL. And then on the next line, we're going to say, what is the API URL? And that's a good question.

What API endpoint is that? That's what we're going to find out next. To do that, we're going to navigate over to the IEX cloud documentations. And the easiest way to find this is just go to IES cloud docs on a Google search.

And it's this first hit here that is not an ad. So click through to this. And what this is, is basically a massive single page resource for all the possible information that you might need about the IEX cloud API.

IEX cloud has remarkably good documentation. There are other APIs out there that maybe aren't as commercialized or are not as mature that won't have documentation that that's this good. So as you kind of look through this, keep in mind that this is probably a better than average documentation page for a public API.

Now, the first thing we need is the base URL for the API. And what the base URL is, is it's basically a URL that will start every HTTP request. And then after that base URL, you have to add which specific endpoint that you want to retrieve from the API.

The way most APIs work is they only expose certain data through each endpoint, which makes things faster if you only need to retrieve certain data. So one example might be, if you have the Google Maps API, you might only want the name of a location. So you would send coordinates and get back its name.

You wouldn't get back other information like population or other sorts of things because you only want name and sending only that limited information is faster than sending information you wouldn't need. So API endpoints are limited in that respect. But to start, the first thing we need is the base URL.

Now, you can see in this first block of text called API reference that the base URL for the API is HTTPS forward slash cloud dot IEXAPIs.com. We're actually using the sandbox mode of this API in this course, which means that we're not charged for any usage, but we get randomized data in exchange. So you couldn't actually use the sandbox mode to create real investment strategies, but it's free and it's still a good way to learn about how data APIs work in general. But because of that, we can't use this base URL.

What we need to do is a control F for sandbox. And if we kind of search through the first little bit, we find this section called testing sandbox. It says this, IDX cloud provides all accounts a free unlimited use sandbox for testing.

(33:59 - 34:46)

Every account will be assigned to test tokens available via the console. All sandbox API or sorry, all sandbox endpoints function the same as production. You only need to change the base URL and tokens.

So what does that mean? We need to change the base URL, which means we need to grab this instead of the base URL that was provided earlier. So copy that to your clipboard, move back to our Jupyter notebook and paste that right there. Awesome.

So we have our base URL. The next thing we need to do is figure out which end point we need. And as you can see, from looking at this, we ideally want to find an endpoint that provides both market capitalization and stock price.

There is one API endpoint provided by IEX cloud that provides both of those. But I'm going to show you first how you would find just any endpoint that provides one of those metrics. And then I'll show you the endpoint we're actually going to use.

(34:46 - 36:31)

So let's start with market capitalization. Note that IEX cloud does have this search bar over here in the top left. But for some reason, I always just find myself using a control F instead.

So what we're going to do here is do market cap, or let's just do market cap, not market capitalization, because it's sometimes abbreviated. So what do we have here for the first? It's enterprise value and price to sales. So these are both metrics that are calculated using market capitalization.

They don't actually give us market capitalization. So neither of those is what we need. The next

one is actually a true market cap metric.

And this is exactly what we need. But if you look through the rest of the response attributes from this API endpoint, this doesn't actually give you stock price. So we, like I said, we ideally want to find an endpoint that has both price and market cap.

And with a bit of searching, you would find that the quote endpoint provides both of those. So I just did a control F for forward slash quote forward slash. And this takes us to the information section about the quote endpoint.

Now, every endpoint description on the IEX cloud API starts with a section on how to execute the the HTTP request for that endpoint. So in this case, we want to execute a get HTTP request. And the endpoint is forward slash stock and then symbol and then forward slash quote.

So if we just copy this and add this to the API endpoint that we have in our Jupyter Notebook, this actually won't work because this curly bracket symbol thing here doesn't mean anything in the context of this string. If we tried to execute this HTTP request as it is, this will return a 400 error probably. So what we need to do is we need to transform this API URL into an F string.

(36:32 - 40:44)

Now, an F string, if you've never used them before, I'll give you a very brief introduction. So if you have string equals free code is awesome. And then you said I need to change the values within this string based on some outside variable.

We will say, we'll create a variable called adjective, and this just describes a word that we want to call free code camp, we will say the adjective is superb instead of awesome. And then we want to pass that adjective variable where this awesome word is within within the string. So the way to do that is you create an F string.

Now, F strings are called F strings because you simply put an F on the start of the string, and then this allows you to interpolate values in. So here, curly brackets, and then we type adjective. Now, what this string actually stores is free code camp is superb.

And you can test that by printing the string. Free code camp is superb. Now, we're going to use this F string functionality to create our API endpoint and we're sorry, yeah, no, to create our API URL.

So what we're going to do is we're going to cut this and just get rid of it. And then here, this already has the curly brackets that are associated with an F string. So all we need to do, and actually, in addition to the curly brackets, it has the right variable name that we called here.

I think I did that by design while kind of building the code for this course. So what we're doing is adding an F there. And then if we print this, if we print this, API URL, this should print out this string, but instead of symbol, it should say APL all caps.

Perfect. That's exactly what we need. Okay.

The last thing we need to do is we need to add an appendage to the end of this API URL that passes in our API token to authenticate us through the IEX Cloud API and basically say, this user does have permission to access this API endpoint. Now the way to do that is you say, question mark token equals, and then we're going to use an F string again and say IEX Cloud token that we imported earlier. If you just type all caps IEX and then hit tab, this will auto complete for you.

Now, if we print this, oh, sorry, I didn't actually have a print statement there. If we print this, this gives us a fully fledged IEX Cloud endpoint that we can ping using the requests library. So moving on to that, what we need to do now is execute an HTTP request and store the results of that HTTP request in some outside variable.

We're going to call that variable data. And what we need to assign it to is a requests dot get method. Now, if you've never used the requests library, that's what we imported at the top of our script here.

It's kind of known as the world's best Python library for executing HTTP requests. Their tagline is their tagline is HTTP for humans. And I quite like that because it does describe well what they do.

So what we need to do here is just type in requests dot get and then pass in our API URL. Now, if we run that, what the heck is this data variable? The easiest way to check is to just do the type function and pass in data. And what it is, is it's a response object, which is contained within the law models module of the request library.

Now, this response object has many different things inside of it. And one of the more interesting ones is the status code. So we can do print data dot status code.

I'm going to hit tab and see if that auto completes. Okay, it didn't set up another it is. And this gives us a 404.

Now, why the heck does it give us a 404? Clearly, I did something wrong here. Okay, so I just figured out the error. And what I missed here was there's an additional kind of suffix that needs to get added to the IX cloud endpoint called forward slash stable.

Now, why does that exist? If you look at this section here, there's two different naming conventions they can use. You can use stable or latest stable is kind of the latest stable API version, as you can see here. Well, latest, you can access the latest API version, which may be in beta.

(40:44 - 42:48)

So basically stable is what you want to use if you want to make sure that your application never breaks. And latest is what you want to use if you want access to the most bleeding edge tech

that might not be fully tested yet. So all we need to do is go back here and here type forward slash stable forward slash and then run this.

Now we get a response to 200 error or a response to 200 code, I mean, which means that the HTTP request was executed properly. And I actually don't like this format. It's not as accessible as this status code attribute that we can access.

And the reason I like this better is you can say, you can test it using equality operators. So is this equal to 200? True. And that's kind of an easy way to handle exceptions with HTTP request.

You say if it's not equal to 200, oops, you could say if it's not equal to 200, then do something. Anyways, what we need to do now is this object is not in any format that we can really access. So what we're going to do is we're going to transform the data that this HTTP request return to us into a JSON object using the dot JSON method.

It actually accepts no parameters. So that's all we need to do. Now if we print the data object or the data variable, it gives us a long JSON type kind of Python dictionary object that we can use to kind of parse objects out.

Now, if you've never worked much with Python dictionaries, they're very useful. I'll make a very quick example here. Dictionary equals now dictionaries are always kind of created using curly brackets.

And what you can say is say a equals sorry, a colon one and B colon two. So this is a Python dictionary called dictionary, very creative naming. The reason why dictionaries are useful is because you can pass in.

So these entries here are called key value pairs. This is the key, this A, and this is the value one. The reason why dictionaries are useful is because you can pass in the key and it returns the value.

(42:48 - 45:17)

Now, this data variable will behave exactly the same way. So as an example, if you want to access this, so this is one key value pair. And if you want to access the value, which is AAPL, all you have to do is pass in the key, which is symbol in square brackets like that.

And this should return apple. Perfect. So the two attributes that we wanted of this are market capitalization and price.

So we're going to parse that in the next section of this course. Okay, so what we're going to do here is say price equals data. And then where is the price contained in this? The easiest way to tell is to do a control F and you can see there's a whole bunch of different price variables here.

The one we want is probably not calculation price, latest price sounds right. So that's the latest price provided by EIS Cloud API. Now, if you ever see something like this and you say, huh, I

don't know what latest price means, that's kind of a bad example.

But let's look at something else in here that's not as obvious. All right, this variable extended change. Now, what the heck is extended change? The IEX cloud documentation should tell you.

So if you just copy that extended change string to your clipboard, and then go back to the IEX cloud docs, run a control F search and type in extended change, it will take you not just to this JSON object, but there should be a description below. So as you can see here, extended change refers to the price change between extended price and latest price. So without knowing what those two means, this is kind of meaningless.

But this was just an example to show you how you can look up the meaning of different data points within the JSON response in the IEX cloud docs. Okay, so that's how to parse price. We can print that out and see what the actual price is.

5 1528. Now, I actually know I'm not sure if the IEX cloud price data is randomized in their sandbox mode. You can test it pretty easily by just doing NASDAQ Apple on Google search.

497. Did it ever get to 515? Okay, so I don't think that this price is accurate, but it's actually not far off from the real world price. So this price that is randomized, but it's it's interestingly quite close to the real price.

So that's how you price out price. And then for market cap, we'll just say market cap equals data and then market cap. And similarly, you can print market cap.

(45:18 - 47:34)

Awesome. This gives like a, I think this is, I'm going to divide this by a trillion. Oh, no, that's sorry.

That's only a billion. That's a trillion. So this gives us a market cap of 2.18 trillion, which again, I think it's pretty close to Apple's real market capitalization.

Yeah. So they're at 2.13 trillion. So in both cases, pretty similar, although still not exactly correct.

Okay. So that's how you parse a API call. What we're going to do now is we're basically going to scale up that process to all of the stocks within our S&P 500 CSV and then store all of the responses in a pandas data frame that we will later save into an Excel file.

So that's what's up next. All right. So the first thing we need to do is actually, well, we're going to exit out of this control F box here because it was lighting that up and causing me some eggs.

The first thing we need to do is specify the columns of our pandas data frame. And to do that, we're going to need to know exactly what we're building. So we're going to build a pandas data frame that has a few columns.

It's going to have the ticker for each stock. It's going to have the stock price for each stock. It's going to have the market capitalization of each stock.

And then it's going to have the number of shares to buy for each stock. So let's start by specifying that in a variable called my columns. And this is just going to be a Python list.

And the entries are going to be ticker, stock price, market capitalization. Hopefully I can spell everything right. Number of shares to buy.

Okay, so that's the Python list. And then what we're going to do now is create a blank pandas data frame that has those columns specified. So to do that, we're going to call this variable final data frame, because we're going to, this will be the final output of this program at the end.

And to create a pandas data frame, use the p.data frame class instantiation. And inside of this, we're going to pass in columns equals my columns. Now, you notice that since this is columns, I didn't want to call this columns because columns equals columns is a bit confusing, although I think it would work.

(47:34 - 50:11)

Let's try to guess that does work. I just think it's quite a bit easier to read it to say my columns, it kind of removes that repetition. Okay, so our pandas data frame has been created.

What does it look like? Pandas data frames are, there's kind of two ways to print them as with most things in a Jupyter notebook, you can actually either use a formal print statement like that, or you can just list a variable name as the last line of a code. So this is kind of what it looks like. There's no data in here right now.

So it's kind of empty. But what we can do is just to create a list of a list that's going to be 0, 0, 0, 0. And this is kind of how pandas data frames typically render. So one of the advantages of not using a formal print statement is that when you just list the data frame variable name on the last line of a code cell, it will render in this nice format that kind of allows you to mouse over it and it changes color.

And it's just a little bit nicer to look at them. The alternative if you use a print statement will print like this in just plain text, and it's not quite as nice. So that's what it looks like when you print it with plain text.

I actually, you know, the, I'm not a big fan of it at all. So we'll stick to printing data frames specifically like this. So that's what our data frame looks like.

I'm going to get rid of this line of zeros because that was just to show you how they render. And that's what we're going to be working with moving forward. So what we're going to do now is I'm going to show you how to append these data points to this pandas data frame.

So what we're going to use to do that is the append method. So final data frame dot append.

And then what we need to put in here is a pandas series that lists all of the entries in the data frame.

So I'll just quickly explain what a pandas series is before, a pandas series is before we move on. So if we add back this row of zeros to our pandas data frame like that. So a pandas data frame is a two dimensional data structure, which means it has rows and columns.

Whereas a pandas series is a one dimensional data frame. If you've ever worked much in Python, it's similar to a Python list. And if you've ever worked with NumPy, it's similar to a NumPy array.

It just has different methods and functions associated with it. But within a pandas data frame, every row and every column of that data frame is a pandas series. So in order to append data to the bottom of a pandas data frame, that data needs to be a pandas series.

I'll show you what I mean now. So let's take this row zeros out and rerun this code cell. And then what we need to do in here is create a pandas series with pd.series. And then the pandas series accepts a Python list.

(50:11 - 53:15)

So lots of different layers here, but what we're eventually going to work out to is adding a row to this pandas data frame. So the first thing we're going to add is the name of the stock that we're adding, which was stored in the symbol variable earlier. And then we're going to add our price and market cap variables, which we created earlier price market cap.

And then we're going to add another cell called N A. And what does this N A mean? Well, this last column, the number of shares to buy, we actually can't calculate it until we have pulled in all the metrics for every stock. So what we're going to do is just store it as N A now. And later on, we'll go back in, access that data and change it once we've actually pulled in data for all of our stocks.

So what happens when we run this code cell? Oh, we missed a comma there. Now we get an error. Can only append a series if ignore index equals true.

So this is super super common when we're working in pandas. This ignore index equals true. Basically almost always needs to be added to the append method whenever you're appending data to a pandas data frame.

So this is what we get. Whoa, this isn't what we wanted. And as you can see all of these columns that we created to have N A N values in them, which means not a number.

And then over here, we have the data that we actually wanted. Now, why is that? It's because we didn't tell the append method, which columns to add that data to. So what we need to do to fix that is we need to add another argument here that says, index equals my columns.

Oh, I missed a comma, I think. No. Oh, so the, sorry, the problem here is that this index equals my column needs to be within the pandas series and not within the pandas data frame.

So small difference there. So when we run this, we're missing a comma again, when we run this, this actually generates what we wanted. So to give a quick recap, we created a final data frame dot append method.

And then within that final data frame dot append method, we created a pandas series that has all of the data points we wanted. And then we specified index equals my columns, which tells the append method, which columns to add this data to. And then the last thing we did was we added ignore index equals true, which is kind of necessary whenever you're appending data to a pandas data frame.

All right, so what we need to do now is loop through every ticker in our stocks variable and execute an API call for that stock and then store the results of that API call in our pandas data frame. So to start, we're going to overwrite our old pandas data frame variable with an empty pandas data frame that has the same columns as before. So columns equals my columns, then we need to create a for loop that says for stock, stock in stocks, ticker.

(53:16 - 58:59)

And to start, let's just print all of these print stock. What happens? Great. So we're successfully looping through all the stocks in that data frame.

We can get rid of that print statement. Now what happens next? We, we need to actually create an API call for each stock. So to do that, we'll scroll up to our old API call cell, and we're going to copy both of these lines, copy, and then scroll down.

All right, so both of those are getting copied in here. This needs to get indented. And then instead of symbol here, we're going to write stock.

Our IDX cloud API token is the same. The data is the same. So that's all good.

Now, one thing to note about looping through our list of stocks in this way is that it's going to be really, really slow. And the reason why it's slow is because executing an HTTP request is one of the slowest things that you can do in Python. So this line here where we actually execute the request is very, very slow.

Later on, we'll see how to improve the performance of this code by doing batch API requests that make it very easy to retrieve information on multiple stocks with a single API requests. But that's kind of a more advanced topic. So we're going to go through with single API requests first.

Okay, so now we need to basically recreate the append statement that we used earlier. So final data frame dot append. Inside the append statement, we need to create a panda series.

Oh, sorry. pv dot series. And inside that panda series, we need to create a Python list.

So lots going on here. The first thing that we're going to add is stock. Now what is stock stock is the incrementer of this for loop.

So this will actually be the ticker of the stock we're working with. The next thing we need to add is data and parsed to latest price. So this is the same parsing that we did before.

This will give us our stock price, which is the second column of the data frame. The last thing is data and then market cap, which is the market capitalization just like we did earlier. And then just like before, we will do n a for the last row.

Within this panda series function, we need to add index equals my columns. And then outside of the panda series as before, we need to add ignore index equals true. Now, since this is going to be so slow, like I mentioned, let's just do this for five stocks and see what happens.

Oh, there's a missing comma right there. Run. Great.

Now if we create final data frame, what's going to happen? Do you think it will have information for all five stocks? And nothing's there. Why is that? Now, the reason why that is is because this final data frame dot append method doesn't actually modify the original data frame unless you tell it to. So to do that, the easiest way is to just say final data frame.

I can't spell today final data frame equals equals and then the append statement. So if we run this and then run the following cell, we should see a different output. Awesome.

So this is for the first five stocks in our list of stocks. If we take off this little argument here, this will take a very long time to run. Let's just see that in action.

I'm going to run this print statement too. And then while this runs, I'm going to have a little sip of my coffee and cut out the rest of this until it's done. All right.

So as you can see, this is a very long pandas data frame that has all of the information we needed. Since it took so long, we're going to move on now to using batch API calls to improve the performance of this code. Now, IEX cloud, like most data providers actually gives you discounts if you use batch API calls because it's a lot less load on their infrastructure.

So that's one reason to use them. The fact that it speeds up your code is another reason to use them. Overall, if you can use batch API calls in your scripts, it's generally a good practice.

So pay attention to this section because it's really important. IEX cloud is a data provider limits their batch API calls to 100 tickers. So what we need to do first is find some way to split our list of tickers up into sub lists of length 100.

Now, it's not super intuitive on how to do that. But basically what I did just to show you is I said how to split a list into sub lists and in Python. Awesome.

Here's the sorry, that took a bit of time. I had to skip ahead, but here's the function that I was talking about. So it's called chunks and in the finished files of this course, you'll see that actually gave credit to this.

It has 3000 uploads, which is pretty crazy even for Stack Overflow. So this is what we'll be using to split our list or our panda series in this case into chunks of size n. So we'll run that code cell to define this function. Okay, what we need to do now is use this chunks function to create a list of lists where every list is no longer than 100.

So to do that, we'll say symbol groups, that's what we'll call this equals chunks and then the list we want to split is stocks index ticker. Now, the reason why we have to pass this as ticker is because this is a panda's data frame and ticker is the header of the only column. So if we just did stocks, this gives us the data frame.

If we do stocks at header ticker, that gives us a series which we can pass into this chunks function. And then we want to say 100. So this gives us a chunks generator.

And to get the actual lists, we just have to pass it into a list function. So we'll do that and then we'll say symbol groups, see what that gives us. Okay, so this gives us a list of lists or more specifically, it gives us a list of pandas series.

(59:00 - 1:01:20)

So this is from index zero to index 99. This is from index 100 to index 199. This is from index 200 to 299, 300 to 399, 400 to 499.

And at the end, we have this one that's much shorter. It's 500 to 504. And that's because there's 505 stocks in the S&P 500.

And this just gives us groups of 100 at most. And since this one only has five minutes shorter. Okay, so moving back up, what we need to do now is make a for loop that lists loops through every list or sorry, every pandas series within that list, executes a batch API call.

And then for every stock in that list, appends the information from that stock to our final data frame. So first thing to do is for I in range, zero to length of symbol groups, and then print. So what does this give us this gives us 012345.

Now, those are all the indices of this symbol group groups list. So if we do this, oh, sorry, that's not what I want to do print that. This will just print out all of the lists within our list of lists called symbol groups.

So now what we need to do is we need to transform all of the stocks that's in each of those lists into a string. And that string will be passed into the URL of the HTTP request that we're executing. So first we'll just create an empty list called symbol strings.

And what this says is it's going to be a list of strings where each string is a comma separated

string of all the stocks in this object. So what we need to do is use the append method to add these in. So we're going to do symbol strings dot append.

And then what we're going to use is the join method called on the comma character join, and then symbol groups. Now, if you've never used the join method before, basically what this does is it says, take all of the elements of this and join them together by separating them with this. So to see that in action, we'll just do print, symbol strings, strings at index i. All right.

So here's one example. Here's another example. And you can tell these examples apart because at the end of the line, there's a gap.

(1:01:20 - 1:02:34)

Sorry, this is one example. I had one extra line there. You can see this example ends there because there's a gap and it starts on the line after this, because this is the last line of the first example.

So there we have our five lists of symbol strings. I'm going to comment this out because we don't want that printing every time we run this code cell. The next thing we need to do is create a blank final data frame again.

So we're using the same variable name, which is probably not a good practice, but we're over writing it each time. So equals p dot data, frame, and columns equals my columns. You guys know the drill by now.

Now let's just print that to make sure that it looks okay. Awesome. So we can delete this out.

Okay, the next thing we need to do is loop through every string in our symbol strings object and then use that string to execute a batch API request. So we're going to use a for loop for this. And to keep the, the nomenclature pretty simple, we're going to just say for symbol string in symbol strings.

(1:02:35 - 1:03:22)

And then we can actually print these out to make sure that this is working right. Single string. Awesome.

That looks good. We're going to take away that print statement now. And then we need to create a batch API call URL.

And as before, this is going to be an F string where we pass in the, the base URL of the API endpoint, and then pass in all of the strings through for each of these symbol strings into the endpoint to get 100, to get data for 100 stocks back. Now, if you've never run a batch API call through IEX call before, you probably are unsure how to do this. Let's just go to the docs here and do a control F for batch and see what comes up.

(1:03:23 - 1:03:35)

Batch requests. Okay. So so what it says here is that use market to query multiple symbols.

(1:03:36 - 1:05:31)

Okay, so this is what we want this stock market. And then as you can see, you pass in one of the parameters as symbols equals a comma separated value of symbols. And then types equals a comma separated value of API endpoints that you want to hit.

And then if applicable, range tells you how much data to pull for. And then at the very end of this, you would pass on your token as well. Okay, so we're going to do is oops, did not mean to click that.

What we're going to do is copy this, go back to our Jupyter Notebook. We're going to pass that in here for now. And then we're going to scroll up to our last API call and get this base URL, including the stable that I missed earlier.

We're going to copy this here. Going to delete the double forward slash, and then we're going to move over to the end of the API call and change a few things. So the first thing we're going to do is remove this last parameter and this range parameter.

We don't need those for the API call that we're doing. The next thing we're going to do is remove the chart and news endpoint from that example, because we don't need them. Next, we're going to actually remove these symbols that are hard coded into this, we're going to do a variable interpretation for symbol string.

And then what we're going to do is move to the very end. And as before, we can just copy on the IEX cloud API token that we had in our original API call. So copy this, scroll back down, go to the end of the string and add it.

Awesome. Now let's just do for symbol string and symbol string up to, but not including the first one. And then we're going to just print this and see what it gives us.

Oh, and I spelled batch wrong here, which I'm going to now. And then print batch API call URL. Now we're getting an error batch API call URL is not defined.

(1:05:32 - 1:06:50)

There's three L's here. That's why run that again. All right.

So this gives us a long URL that looks okay to me. One really easy way to test this is to just click on it. It's a 400 bad request.

I'm not sure why stable slash stock slash batch. Let's make sure that that matches slash stock slash market slash batch symbol symbol string. Oh, so the problem here is that when you're

passing on multiple parameters to an API request.

So this first character is a question mark. And then every successive character that chains these parameters together needs to be an ampersand. So we're going to copy this ampersand and replace this question mark with it.

And I think that should fix the problem. Great. So this is actually opened in a browser, the JSON that is going to be returned from this API request.

So now we can do what we did before use the requests library to get this data. So we're going to data equals requests dot get and then that URL. And then as before, we're going to pass the JSON method onto this.

(1:06:50 - 1:09:23)

Actually, before we do that, let's just do print data dot status code. So it returns 200, which means it's working properly. Now what we can do is pass the JSON method onto this data variable.

And then we need to parse data from this. So more specifically, we need to parse data for every specific stock that's in this symbol string. Now the way to do this is just like before how we use this join method to join together all of the different strings that were contained in that list, you can use the split method to basically do the opposite.

So we can say for symbol in symbol string dot split, and we're going to split on the comma character. Let's just print every symbol and see what happens. Awesome.

So it looks like it's successfully looping through all of the first 100 stocks there. And what we need to do now is parse the bash API call for data for every specific stock in the order that we need, and then append that data to the pandas data frame. So just like before, we're going to use the append method, we're going to say final data frame equals final data frame dot append.

And then in here, we're going to specify that it's a pandas series. And inside the pandas series, we're going to pass in a Python list. So all of that is exactly what we did before.

So I kind of skipped over it. And then here, we're going to pass in the same four variables. So symbol, and then price, which will be data.

And then we have to do actually multiple levels of parsing this time, because not only do we have to parse the metric for that stock, but we also have to parse the batch API call to get the information for that specific stock. So here's how you do that. We do simple and then quote.

So this is the stock and then the endpoint. And then we need to parse out the metric, latest price. And we can follow similar logic here to get the market capitalization.

So market cap. And then just like before, we are going to not specify the last column, we will calculate that later. Let's run this and see what happens.

Actually, I'm going to add one last line of this that prints out our data frame. All right, let's run this and see what happens. Type error can only append a series.

(1:09:24 - 1:10:26)

This is that ignore index specification that I mentioned earlier. So that needs to get added in right here. And actually, now that I think about it, inside this pandas series class instantiation, we need to pass in index equals my columns.

All right, let's try this. Invelds index index and ignore index equals true. I'm missing a comma there.

One more time. Awesome. So as you can see, we've successfully appended the ticker, the stock price, the market capitalization and the number of shares to buy.

The only other change we have to make to this code cell is to remove this little appendage right here that made it loop only over the first instance. So now when we run this, it will actually loop over every symbol string and add every stock within our list to our pandas data frame. So let's try that.

Awesome. As you can see, that was way, way faster than using individual API calls for each stock. The other one took probably two or three minutes and that was only a few seconds.

(1:10:26 - 1:11:22)

So huge improvement there. I mean, in theory, if all the API requests take the same amount of time using batch API calls should make it 100 times faster because it does one call for every 100 stocks. But anyways, much faster than the case.

And it looks like we go all the way from A to Z. All the data looks okay. So we're ready to proceed to calculate the number of shares to buy. All right.

So the reason why we left this calculating the number of shares to buy step until later in the script is because I want this to actually work regardless of how big your portfolio is. So how we're going to handle that is we're going to create a Python input that says how large is your portfolio and then based on what you tell Python, it will calculate the number of shares to buy accordingly. So to do that, we're going to use Python's input function.

And more specifically, we're going to say portfolio size is equal to input. And then inside this input, it accepts a string and that string is kind of what question you want Python to ask you. So here's how it looks.

(1:11:23 - 1:12:52)

Enter the value of your portfolio. Now, if we run this, you can see there's an input that says enter the value of your portfolio. We can write, you know, $1,000.

And then we can access that value later on in the script. So now if we do, you know, portfolio size, it'll print out a thousand note that this is actually a string. So that fact that this accepts a string is actually something we need to make sure we handle because since it's a string, you could say, I don't have a portfolio.

And that would actually kind of break the rest of the script because it's going to try to do mathematical operations on this string, which is obviously impossible. So what we need to do is create kind of a try accept statement. So if you're not familiar with try accept, basically the way these work is try accept, you could say, try to do this.

And if this doesn't work, it will do whatever specified here. So one example would be like if you say, two plus n, n is not defined. So then it will print out, you know, n is not defined.

We're going to comment this out so it doesn't run. Now if we run this, it should print n is not defined. Awesome.

Now if we say two plus two, it will doesn't print it. But if we wrapped it in a print statement, it would for okay, so we're going to use this same logic to handle the fact that people might not put number variables into this portfolio size input. So the first thing to do is uncomment this.

(1:12:53 - 1:19:59)

The next thing we need to do is to try to kind of handle our input as a float variable. So to do that, we're just going to create a variable called val. And we're going to say that's equal to float portfolio size.

Now this float function is basically going to take this portfolio size variable and try to force it to be a float. So if we say print val, actually no, ignore that. What we're going to do is put that in here and say print val.

And then for this except we're going to say, please enter an integer. Now if we put in a sentence, we'll say again, I don't have a portfolio. It returns, please enter an integer.

Now this broad try accept statement is actually not good. And the reason why is because this will react regardless of what type of error is happening. So what we need to do to make sure that it only handles the specific type of error that we're trying to address is we need to try to do this and kind of force an error.

So put a string in there. And then as you can see, the type of error that's created is a value error. So we need to basically change this script so that it only reacts to value errors instead of reacting to all types of errors.

And the way that you do that is you put the type of error right here. You can run this again and

see that it does handle this properly. Okay, so what we want to do now is instead of just handling this, instead of just handling this and saying please enter an integer, we need to actually make it redo this.

So how does this look? Please enter an integer. What we want to do is say in here, enter the value. And then above this, we want to tell them exactly why they're getting this input prompt again.

So we're going to print and say that's we'll use double brackets. That's not a number. And then four slash n. So that'll create a new line.

Please, please try again. And then val equals float. Awesome.

So just to recap, we try to get the value of someone's portfolio using the input function. In the first step of the try accept block, we try to force that that that inputted value to be a float variable and assign it to a variable called val. If that doesn't work because of a value error, then we print out that's not a number, please try again.

And then we prompt them to enter the value of their portfolio again. And then we, we force it to be a strongly type float again. Now, one interesting thing to note about this is that if you do it twice, string and then string, it will still trigger a value error.

So you need to kind of rely that the user of this script is going to be like, I guess astute enough to not enter a string twice. So it might look something like this, they go and say, enter the value of your portfolio $1,000. And they say, that's not a number, please try again.

So then you say, Oh, it's $1,000. And then you can proceed to the rest of the script. Okay, so now that we've accepted the number from the user of this script, we need to actually use that number to base our share calculations on.

Now, since this is equal weight, every stock in the portfolio will have the same position size and position size is kind of just a fancy financial term that says how much money you're going to invest in each stock. Now, since there's 505 stocks, an easy example would be if your portfolio is $505,000, all you would do is invest $1,000 in each stock. So as you can see, the easiest way to calculate that is just to say position size is equal to value divided by the length of any column from the final data frame.

So in this case, we'll just do final data frame dot index and then print position size. So this says we made a small portfolio here of $1,000. So it's just saying give us $1.98 in each stock.

Let's use something more reasonable like a million dollars. Awesome. So now it's saying invest $19,000 in each stock.

That sounds about right to me. Oh, hang on. Is this a million or one, two, three? That's 10 million.

So $19,000 in each stock or almost $20,000 makes sense for a $10 million portfolio. All right. So we have the position size now.

Now we need to calculate how many shares of each stock need to be purchased to get to that position size in that stock. So let's take an easy example. Apple stock price is about $500 right now.

So if we wanted a number of Apple shares to buy, you would just say position size divided by Apple stock price, which is $500 ish. And then you can print the number of Apple shares. And in this case, it says, okay, buy 39.6 Apple shares.

Now, this introduces an interesting concept in that a lot of places do not support fractional trading, which means that you can't buy a portion of a share. You can only buy whole shares. So you can buy 39 shares or you can buy 40 shares, but you cannot buy 39.6 shares.

So how do we handle this? Well, it's tempting to just say, let's round it to the closest number up or down. But what might happen then is that if you are rounding up more integers than you're rounding down, you'll actually end up buying more stock than you have money for. And that is not a good thing.

You would get to the end of your allocation and say, oh man, I don't actually have enough money left over to get to my position size target on this stock. So because of that, we actually have to round each of these down. And that's where that math module that we imported earlier comes in.

So math dot floor is Python's round down function. So if we print this, it should give us 39. Oh shoot, math is not defined.

I think it might be lower case. Yeah, math dot floor. So that gives us 39.

Okay, so this is kind of like the logical intuition on how to do this. We now need to programmatically loop through every row of our pandas data frame and actually apply this logic to the number of shares to buy column. Now to do this, I'm just going to comment that out because it'll be useful for us later.

All right, so we're going to say for I in range from zero, and then we needed to go to the last row of the data frame and the easiest way to do that is just say the length of any row within the data frame. And what I typically use is final data frame index. As you can see, this logic is actually basically what I used here as well.

(1:20:00 - 1:20:55)

So let's use this and then just print everything in here. All right, this goes from zero to 504. Let's just make sure that adds up with the data frame we clicked earlier or printed earlier, I should say it goes from zero to 504.

Awesome. Now we need to access this cell in each row of the data frame. The easiest way to do that is using the LOC method.

So what we're going to do is say final data frame dot LOC and this is basically like an easy row column way of accessing data in pandas. So for the row, we're going to say I and for the column, we're going to say number of shares to buy. And let's just print this to see if it prints a bunch of NAs because that's what it should print.

Awesome. So we're correctly accessing the data. Now we need to actually assign the data.

(1:20:55 - 1:22:35)

So to do that, you can just use the equality operator and then say math dot floor. And then we need to do basically this calculation here position size divided by the stock price. So we'll say position size divided by now, what are we going to do for stock price? Do we have to execute another API call and pull that data from IEX again? We actually don't and the reason why is because that data is already stored in the pandas data frame.

So we can take similar logic to this LOC method here and put that there, paste it and just change this to stock price. Well, this tab autocomplete is not working. So if we run this, what happens? It ran correctly.

Let's do this again and print the data frame to see if it looks okay. Awesome. So as you can see it correctly kind of shows the number of shares to buy here.

One easy way to fact check this. Well, there's a couple things you do. The first thing is to say, okay, this Apple stock price is close to the 39 that we calculated earlier.

So that's a good sign. Another thing you do is say, look at two stocks that have different stock prices and the one that has the higher stock price should be recommended to buy less shares than the one that has a lower stock price. So let's just use the first two rows of this data frame.

As an example, you can see that for this stock with ticker A, it costs $100 to buy it and you're buying $197. And then the second stock has a much lower stock price. And because of that, you have to buy many more shares to actually reach your target position size.

(1:22:35 - 1:29:00)

Now that kind of all looks good. So we can move on. I'm going to take away this comment code and just tidy this up a little bit and we can proceed to the next cell.

All right. So this brings us to the last section of this project, which is to save our pandas data frame into an XLSX file for non technical users to access through Excel. So the big idea here is that we did all of the work in a Python script.

You could schedule this script to run periodically. And then the output would be sent to your

trading team to actually buy and sell stocks in response to the output. So to do this, we're going to use the XLSX writer library for Python.

Now, oops, the new environment. If you remember, at the start of this project, many lines of code ago, we actually imported this here. So it's actually in our script already, we just need to use it now to save our data frame as an Excel file.

Now, XLSX writer is an excellent package and offers an insane amount of customization. But the trade off for this is that the library can seem a bit complex to new users. I'm going to do my best to explain well throughout this section.

And because of that, this might be a bit long. So if this seems easy, or you use that, if this seems easy, or you've used XLSX writer in the past, you can feel free to speed up the speed here a bit or skip ahead. Alright, so the first thing we need to do is actually initialize our XLSX writer object.

And the way that we do that is by saying writer equals PD dot Excel writer. And then here, we need to pass in the file name that we want to save to in this case, we're going to do recommended trades dot XLSX. And then we need to specify the engine, which is XLSX writer.

Now, let's break this down a little bit to kind of describe what's going on here. As you can see, the class that we're initializing here is from the pandas library and not actually from XLSX writer. And the reason why that is, is because since pandas deals with tabular data, and so much tabular data is actually saved to Excel files, they have a very tightly coupled integration.

And it actually kind of is easiest to initialize a new writer object from pandas rather than XLSX writer. And kind of by default, you always save these objects to a variable called writer. It allows you to reuse your code without having to change the object name later.

And it's kind of just the best practice. So we're going to say writer equals p dot Excel writer. And then this is the file name that we want to save to and then the engine equals XLSX writer.

It may seem a bit redundant, but this library can also be used to save XML file. So we have to actually specify that we want to work in Excel. So that's the first thing we need to do.

The next thing we need to do is actually to pass our pandas data frame into this object in specify which tab of the Excel file we want it to be saved to. So to do that, we do final data frame dot to Excel. And then the first thing we pass in is that writer object.

The next thing we pass in is the name of the tab we want it to be saved to. So we'll say recommended trades again. And the last thing we pass in is an index equals false argument.

Alright, so our object has been created. The next thing we need to do is to create the formats we need for our XLSX file. Now earlier when I said that this library can be confusing or complicated to new users, this is where it gets really, really complicated.

Formatting Excel files with XLSX writer is a science and there's a lot to it, but I'm going to try to keep things simple here and hopefully you guys won't run into any hiccups. The first things we're going to do is create, I guess two variables that specify the color scheme for our Excel sheet. And then we will reference those variables in all of the formats that we create later on.

So that if we want to change one color, it will be reflected across all the different formats that we create later instead of hard coding them in and having to change many different instances of a specific color. So specifically, what we're going to create is a background color variable and a font color variable. So background color.

And these are just going to be empty strings for now, font color. Alright, so what goes in here, these actually store the HTML hex codes for the very for the colors that we want to select. So I'm going to use two colors that kind of match the style of the free code camp website.

I really like it. It's command line chic and very modern. So we're just going to follow that to make things easy.

The background color is going to be 0A0A23 and the font color is going to be F6F. Alright, so that's that. Now we need to actually create a few different formats that we will be referencing when we actually apply the formats to our cells of our Excel sheet later on.

So as you can see up here, we need a string format, a dollar format with decimals, a dollar format without decimals and an integer format. So let's start with string format. And then what we're going to do here is create a writer.book.add format method.

Inside this add format method, we have to create a dictionary and this dictionary specifies the format of the cells that are going to have this applied to them later. So what goes in here? There's three main attributes, font color, background color and border. Now each of these should actually be strings.

I'm going to fix that indenting issue. Since this is a dictionary, what is going on? I find the indenting on Jupyter notebooks leaves a lot to be desired. Since this is a dictionary, these should be strings and we have to add colons to each one.

Alright, so the font color is going to be font color. And this font color variable is what we defined up here. This background color is going to be background color.

And this border is going to be one. This just means to add a solid border around each one. Now this is our first one.

Let's run this code to make sure it works properly. Awesome. Now we need to basically template off of this and create a few other formats.

So we have a string format and then we need to name the other ones that we described before this code cell. Say string format, dollar format and integer format. Now all of these are going to

share the same attributes that the string format specified.

(1:29:01 - 1:33:35)

We just need to add a few things to each one. So for dollar format, we need to specify a number format attribute like this. And the way that this works is actually pretty easy.

You just pass in a number with zeros in it, templated the way that you want it to be formatted. So in this case, it will look like that. And then we will do the same thing for the integer format.

So num format. And then we're just going to say zero. Awesome.

Let's run this code cell to make sure there's no syntax errors. Excellent. Alright, so now we need to apply the formats that we just created to our XLSX file.

And this is like a big complicated. So what I did here was I actually provided an example and we can kind of use this to template on for all of the columns in our sheet. So what we need to do is say writer dot sheets and then parse out the recommended trade sheet from that object.

And then we use the set column method on that item to specify information about the column we're doing. Now we're going to do this one by one first and then we're going to kind of put everything into a loop because it'll be a bit more efficient on code and much more readable. So let's start with column A. You actually can't just put A you have to put A through A like that.

So that tells you the column and then we need to specify the column width. Let's do 18 pixels. And then you need to say which format you want to apply to it.

Column A is our ticker format. So we need string format. It's not a number.

Now, if you run that it returns zero. And that basically tells you that it ran properly. The way to actually see this in action is you need to actually save your writer object.

Now to do that you say writer dot save. And then if you go to your working directory where you're running through this you should be able to open up your recommended trades excel file and see that the first column has actually been formatted properly. So let's do that for all the other columns and then we can change a couple of things to make it formatted better.

So how to do this. We'll just copy this a few times and change the letter four times to be specific. So this should be B through B. This should be C through C and this should be D through D. Now let's run this and see that what happens to the excel sheet.

Only one was formatted. Why is that? Oh, I think we have to go back here and re initialize this. All right.

Let's try now. Awesome. So as you can see, the columns are being formatted properly, but the headers aren't just so you know, we're going to solve that later.

It's not something that I'm ignoring, but we will solve that in a few cells. So what we need to do now is we're going to simplify this by making two loops and each of those loops is going to work with this column formats dictionary that we're going to create. So let's create column formats and make a dictionary.

Now what this dictionary is going to be is it's going to have a key for each column. So A B C and D and the value for each column is going to be a list inside that list. The first item is going to be the title and the second item is going to be which format we want to apply format.

So ticker string format and then stock price dollar format. Oh, comma there. Market cap.

And the last one is going to be number of shares to buy and integer format. So let's run that. So that cell ran correctly, which tells you that that dictionary is formatted properly.

And now we're ready to start building a couple loops to do all of the work that we did up here, but automatically. So the loop that we're going to be building specifically is going to loop through every key in that dictionary. So we're going to say four columns in column formats dot keys.

If you've never used the keys method to iterate through a dictionary before, it basically just returns all of these. So you print the column, it should print A B C B. Yep. Now what we need to do is create a writer dot sheets, recommended trades dot set column.

(1:33:36 - 1:34:44)

We're basically doing all the work we did up here. So set column. Here we're going to do an F string that interpolates in column column.

So this would just be like for column A would be a colon A. We're going to MS string and then we're going to do 18 pixels wide. And then we're going to do column formats at column. So this will be this dictionary at key A. So this would return this list.

And then we need the second element of this list. And since Python is zero index, we parse one. Great.

So that runs properly. Awesome. So if you if we redo all of this to okay, here's what we're going to do.

We're going to comment out this. We're going to put a writer dot save method there. And then we're going to go back to the start, reinitialize our writer object and run all of this code.

(1:34:46 - 1:35:25)

Awesome. Let's take a look and see what our Excel file looks like. Great.

So everything's being formatted properly. In that loop, our code is a lot cleaner. It's easier to

read.

It's a big improvement. Now what we need to do is handle these. And columns in XLSX writer are kind of hard to deal with.

What I always do is I just overwrite them. So writer dot sheets, recommended trades. And then I do dot write.

And this write method, you pass in what information you want stored in the cell and what format you want to apply to that cell. You also need to specify the location of the cell. So here's what we're going to do.

(1:35:28 - 1:36:43)

A1. We're going to say ticker and we're going to say string format. Now, if we go back to the top again, reinitialize this and run through the rest of this and go back to our Excel file.

The first cell is formatted properly. So now we just need to do the same thing for stock price market capitalization and number of shares to buy. Let's go back.

Easiest way to handle that is to just copy and paste. So this will change to A1, B1. No, sorry, that should be A1 or sorry, A2 or B1.

Yeah, B1. Okay, perfect. All right.

A1, B1, C1, D1. And this will be stock price with dollar format. This will be number of shares to buy with integer format.

(1:36:44 - 1:36:52)

All right. As before, we're going to go back to the top. We're going to reinitialize this Excel writer object, run our code to the very end and then take another look at our Excel file.

(1:36:53 - 1:37:18)

Awesome. So everything looks formatted properly. The only thing that might be worth improving is to widen this a little bit.

But I mean, anyone who has an auto format or for Excel will be able to handle that pretty easily. All right. So this violates the programming principle of do not repeat yourself.

This is very, very loopable. So we're going to create another loop here that basically does this logic. So all right, so we're going to add this in here.

(1:37:19 - 1:37:34)

And what we need to do is say column here and convert this to an F string. So as this loops

through, this will go from A1 to B1 to C1 to D1. And then we need to capture this value here.

(1:37:35 - 1:38:30)

So it's going to be column, column, format. So this is the dictionary. And then we parse out columns.

So this is the value of a key value pair. And then we want zero. And this will be the first entry of the list that is the value in the key value pair.

So that's this. And then this needs to get replaced with the second entry of the list. Easiest way to do that is to just copy this and replace the zero with a one.

So let's go there. One. All right.

As before, we're going to scroll all the way up to the top, reinitialize our writer object, move all the way through to the end. Let's open this up and see if everything looks okay. Great.

So this is the final output of the first project from this algorithmic trading and Python course. It says here that the last step is to save the writer output. That's actually this line.

(1:38:30 - 1:39:43)

So we're already done. And just like that, you build your first algorithmic trading project in Python. I hope you had fun.

The next two projects are going to be even more awesome and you're going to learn a lot more. So I hope you enjoy the rest of this course and kudos to you for sticking through it so far. All right.

So welcome to the second project for this course where we're going to build a quantitative momentum strategy. What we need to do to start just like before is we need to open up our Jupyter here in my terminal in my home root directory. And what I need to do is navigate into my dev folder.

And within that dev folder, I need to navigate into algorithmic trading in Python, which you can see here, autocompleted for me. So we'll move into there. And now that we're here, we need to activate our virtual environment.

So you'll remember from before that the command to do that is source and then V and V forward slash bin forward slash activate. If you look to the left of your terminal command here, you can see that that V and V in parentheses there shows that the virtual environment has been activated correctly. And now that the virtual environment has been activated, you can launch it with Python dash M notebook.

That will launch your Jupyter notebook. Now, one thing that I'm not sure I mentioned before is

that you can deactivate a virtual environment by simply typing deactivate. And if you try to launch your Jupyter notebook without this, it will actually fail because it'll say no module name notebook.

(1:39:43 - 1:41:04)

So we need to reactivate our virtual environment and then launch your Jupyter notebook. This will start the server in our browser and we can get started. Just like before, we're going to want to navigate into starter files and then open up the second project this time, which is quantitative momentum strategy.

So let's open that to get started. All right, so just like before this Jupyter notebook starts with a bit of background information. So it says momentum investing means investing in the stocks that have increased in price the most.

For this project, we're going to build an investing strategy that selects the 50 stocks with the highest price momentum. From there, we will calculate recommended trades for an equal weight portfolio of these stocks. So this project will combine a lot of the stuff that we used in the last section.

So that repetition will help you remember what we actually just did because I know that that last project actually contained a lot of stuff. And then we're going to build on top of that by not just equal weighting a already selected universe of stocks being the S&P 500, but we're going to actually select a subset of those stocks based on their momentum characteristics. So let's get started by first importing some libraries.

All right, so just like before, we're going to need NumPy and we're going to import that as NP. We're going to need pandas, and we're going to import that as PD. So as a quick recap, NumPy is the best Python library for numerical computing and pandas is a data science library that makes it very easy to work with tabular data in Python.

(1:41:05 - 1:42:19)

You'll remember that we worked a bunch with the PD.data frame structure in the last project, and we'll do the same thing in this project. And we're also going to need the requests library for making HTTP requests. We're going to need the math library for performing some basic math functions.

And we're going to need the SciPy library. Actually, we're not going to need the whole library. We're just going to need the stats library from SciPy import stats.

And the last thing that we need is the XLSX writer library, which we saw in the last section allows us to easily format and save Excel files from a Python script. Now we've seen all of these before, except for the SciPy stats module. And what this does is it makes it very easy to calculate percentile scores.

So we'll see later on in this project that what we're actually going to do is we're going to gather and parse momentum metrics for all of the stocks in our universe. And then we're going to calculate percentile scores for those momentum metrics. Then we're going to rank the stocks on those percentile scores.

And we're going to select the 50 stocks that have the highest average percentile score across a broad basket of momentum metrics. So if you haven't seen SciPy stats, it's pretty easy. We're just going to use the percentile score calculation feature of it.

(1:42:19 - 1:42:27)

And yeah, so that's it for our libraries. Let's move on to importing our list of stocks. So actually, we should run this code cell first.

(1:42:28 - 1:43:57)

All right. So now that our imports have been done for our open source Python libraries, we need to actually import our list of stocks. Now we also need to import our API token.

We did both of these steps in the last project as well. So before I actually code this out, try to go through and without looking at your last one, figure out how to import your list of stocks into a pandas series and figure out how to import your API token from your secrets file. So cut here and try to do that before proceeding.

Okay, I hope you had success doing that. If you didn't, the command to import your list of stocks is pd underscore, sorry, dot read underscore csv and then type in your list of stocks. Now this should auto complete if you just type in sp and then you can print it out to make sure it looks okay.

Just like before it has all five hundred and five stocks in the S&P five hundred and then for the secrets file we want from secrets import and if you hit tab, you can see that the IAS cloud API token actually shows up here. So that is great. We can run this code cell to get both of those imported and we can move on to make our first API call.

Now this is where this project will start to be quite a bit different than the last project that we worked through because we're actually going to be pulling different stock market information from the IAS cloud API. More specifically, instead of trying to pull price and market capitalization, we're going to be pulling price and one year stock returns. So to do that, we're going to go to the IAS cloud docs.

(1:43:58 - 1:48:18)

Oops, let's try and keep this all in one browser window. There we go. So IAS cloud docs and we're going to go here.

It's usually the first non add entry in a Google search and then we're going to run a control F

and see what kind of a return metrics we can actually find in here. Okay, so return is clearly not a good thing to search for because it gives 341 hits and you can see why that is. It's because a lot of these will say most endpoints return.

So the fact that endpoints can return something and stocks can also return a percentage is not good. What if we just type in momentum? These are all more complex momentum metrics than we need. Okay, what if we type in price return? Zero.

What if we type in price return with no space? Okay, what if we type in year return? This might give us an API endpoint that gives us a one year return or something like that. Still nothing. What if we do performance? Okay, this gives us 21 hits.

Sector performance. As you can see, it's not always super straightforward on how to find these. Okay, so now I'm searching through return again.

It is a lot of hits, but I feel like it's going to be near the top of this documentation page. Okay, so this looks promising. This is a historical prices endpoint and this would actually allow you to get the job and you could pull in historical prices for let's say one year and use the today's price divided by the price one year ago and then subtract one and that would give you your one year price return.

However, I know that IEX actually calculates this metric in a separate endpoint. So we're going to keep moving through this until we find it. All right, so I had to cut there turns over.

We're actually looking for is the word change, not the word return. And I do a control that if it's the first thing we find, of course. So this is the key stats endpoint, which you can get with the stats keyword in the URL.

And if you look down here, there's actually an awesome group of momentum metrics. So you can do maximum percent change, five year percent change, two year percent change, one year, year to date, six months, three months. In case you can't see this, I'll zoom in a little bit.

I guess you can't zoom. Oh yeah, there we go. That's a lot of zoom.

There we go. There we go. All right, so max change, five year, two year, one year, year to date, six months, three months, one month, three days, sorry, 30 days and five days.

So this is the stats endpoint. What we're going to do is we're going to copy this and we're just going to put it in our Jupyter notebook for now to come back to later. And we also need the sandbox base API endpoint.

I'm going to zoom back to normal zoom because it's a little easier for me to navigate. So here's the testing sandbox. We're going to find that this is the base URL.

We're going to put this in here right here. And then remember from last time, we need to actually include stable here. So this is a example of the base URL that we need to hit with our

API call.

(1:48:20 - 1:48:40)

We're going to call this string API underscore URL, and then we actually need to specify a symbol. So just like before, we're going to use apple as an example symbol, and we're going to use an F string to interpolate that symbol variable into this. We can take this right off the end because we're not actually trying to parse a specific stat.

(1:48:41 - 1:49:16)

And then we can use the requests library to make an easy API call to get this data. So to that, we're going to say data is equal to requests dot get. And then we're going to do API underscore URL.

And then we're going to print the status code of that HTTP request to make sure that went through properly. So this gives 400. We did something wrong.

What is it? Sandbox that is, oh, we didn't pass on our IEX cloud API token. So we need to do question mark token equals IEX and then tab it should auto complete. Try that.

(1:49:17 - 1:49:56)

Alright, so returns 200, which means the API call is working properly. Let's transform this into a JSON and print this again. Alright, so as you can see this can as you can see, this contains all the information that it should contain.

The information that we specifically want is the one year price returns for every stock in the universe. So to do that, it's actually quite easy. We're going to cover that next.

So to parse a JSON object like this, we saw in the last section that you just pass in square brackets and then the key of the dictionary. So the key that we want now is one year change percent and all these keys generally don't start with integers. They start with letters.

(1:49:56 - 1:53:08)

So that's why it says year five instead of five year and etc. But this is the key we want. So we're going to copy this.

We're going to move down here. We're going to add that in and then this should give us one point four two. So what does that mean? That means that Apple has increased in price one hundred and forty two percent in the last year, which is pretty impressive.

It means that the stock has more than doubled, but I guess that's what happens when you sell a lot of expensive iPhones. So just like before, we're going to kind of loop through all of the stocks in our universe and provide API calls for all of them. Instead of living through them one

by one, like we saw in the last section, which is extremely slow, we're going to move right on to executing batch API calls because that's kind of a better practice and there's no point practicing executing single API calls when you'll generally never do that in practice.

So the first thing we need to do is chunk our list of stocks into groups of one hundred using the chunks function that we created last time. So to make this easy, I've actually included a bunch of reusable code from the last project here. So just run this code cell.

That will give us the chunks function. And then we use the chunks function to divide our list of stocks into symbol groups of length one hundred. Then we create an empty list called symbol strings and then for every one hundred stocks in this symbol groups list, we create a comma separated list of strings and then add them to this symbol strings variable.

So if you go through every symbol string in that list, we did this last project as well, but we're going to do it again just for practice. And then you print it. You'll see that this is basically just five lists and I'm sorry, this should be four in.

Yeah. So this is just five lists of length one hundred. So when I say lists, they're not Python lists, they're actually strings separated by commas, but there's five of them.

So here's one, here's another, here's where the third one ends, fourth one, the fifth one. And I said there's five, there's actually six, sorry. That's because we have this short one of length five at the end.

Okay, so that's all of this. I'm going to delete this out. And we are going to move on to creating a blank pandas that data frame.

So just like before, we're going to call this final data frame, and we're going to instantiate it by saying PD dot data frame, and then columns equals, and then you'll see up in this code. So I've actually defined the names of the columns we want here. So it's ticker price, one year price return and number of shares to buy.

These are basically the same column names as we used in the last project, except market capitalization has been replaced by one year price return here. So we can go down here and specify those columns by saying columns equals my underscore columns. And then if we print this, we should see an empty pandas data frame with the columns we specified.

Awesome. So that's good. We're going to delete that.

And then we need to loop through all of our symbol strings in the symbol strings object and create a batch object also for symbol string in symbol strings. And then we want to create a batch API call URL. We're going to leave that as an empty string for a moment.

(1:53:08 - 1:54:07)

And then we want to create a data variable and that's going to be a request dot get method

that accepts batch API call URL and then calls the JSON method on it to transform it from a requests object to a JSON object. So what goes in here? Let's go back to the IACrowd docs to look at batch API call syntax again. All right.

So I just did a control F for batch here to find this section, which tells you everything you need to know about batch requests. So stock and then symbol and then batch. Actually, we want to query.

So this syntax here is for querying multiple end points for one stop, but we want to query one endpoint for multiple stocks. So we actually want to use this syntax here. So I'm going to copy this link address and I think this just gives us the last bit.

(1:54:08 - 1:56:12)

No, it gives us the entire thing. We're going to want to change this to sandbox because by default, the docs give you a live URL, not a sandbox URL. And then we're going to want to move down here and change a few things here.

So what endpoint do we need first? Actually, to start, move this up here so you can actually see it. To start, let's take off this last parameter and the range parameter. So those can both be gone and then we're going to have to add back our token equals and then IEX cloud.

Oh, sorry, this isn't an F string. So that won't auto complete IEX cloud and yep, IEX cloud API token. Then we're going to want to change the endpoints next.

So what goes there? Let's scroll back up to see which endpoint we actually queried earlier. All right. So we use the stats endpoint.

So let's put that down into our batch API call stats. And then here we want to change our symbols to be an interpolated variable that matches whatever the incrementer is of our for loop. So that is symbol string.

All right. So now what we need to do is we're going to run over this loop once just to make sure that we're getting a 200 HTTP code to make sure that the HTTP request is being executed properly. So to do that, we're going to print data.status code.

And for this to actually work properly, we're going to have to remove this JSON method temporarily from the HTTP request. And then here where we're specifying what we loop over, we're going to want to change this to a colon one, which tells this to loop only over the first entry of that string. So if this is working properly, we'll get a response of 200.

Awesome. So that's good. We can get rid of this status code attribute from the data object and call this JSON method on it to see what the structure of the HTTP response actually looks like.

So this is basically a huge dictionary. It has many levels. So at the top level, there is a key for this ticker and a value of a dictionary.

(1:56:13 - 1:56:59)

And then within that dictionary, there is a key for the endpoint stats and a dictionary for the value. And then within that dictionary, there's a key for every metric with a value that corresponds to that metric. So we have to do multiple levels of parsing here.

But before we do that, we actually have to loop over all of the stocks. So let's let's build that loop first. What we're going to do is we're going to say for symbol in symbol string dot split, and we're going to split on the colon character, which separates all of the tickers in the symbol string.

So let's see what that split method actually returns before we go any further. Copy that, comment that out, and then print that, oops, print that, no, print this is what I want to print. There we go.

(1:57:02 - 1:58:09)

Awesome. So let's run that and see what it gives us. As you can see, this is a Python list where each item in the list is a symbol that corresponds to all the comma separated symbols in the symbol string.

Okay, perfect. So we'll get rid of this. Did not mean to open my dev tools.

We will uncomment this and then we will start parsing the data in this loop. So we're going to say, so what we want to do here is we want to, for every ticker in this list, we want to append the relevant metrics to the final data frame. So to that we're going to say final data frame is equal to final data frame dot append.

And then within this append method, there's a few things we need. We need a pandas series and that pandas series will accept a Python list and it will also accept the index equals my underscore columns argument that, as you'll recall from the first project, this tells the append method where to actually put the new metrics in the existing pandas data frame. And then outside the pandas series, we're going to actually add ignore index equals true.

(1:58:10 - 1:58:15)

Okay, perfect. So this is where we want to do our parsing. The first metric we want is symbol.

(1:58:15 - 1:59:20)

The second metric we want is price. So to get price, we're going to need to do data and then parse the symbol and then parse the stats endpoint. And then we're going to have to actually see what the name of the price metric is within this.

So let's right here, let's just print out one example of this. Oh, sorry. Not right there.

Right here. Let's print data and then we'll use Apple as an example ticker and then we'll use stats and then let's just see what this gives us. All right.

This is giving us an error because we didn't actually format the rest of it properly. But let's look and see here what we have for price. I'm going to do control F for price and see if anything highlights up here.

Let's do some market cap employees. Doesn't look like this actually has any price data. So we need to add here to our endpoints.

(1:59:20 - 2:00:49)

We're going to say price and stats. Let's see what that gives us. If we parse price, I think this just gives us.

Yeah, $504. Okay, perfect. So this is what we need to do for price.

We're going to replace Apple with the symbol that we're looping over and then we're also going to pass in one year price return. So we can use this print statement up here to see what we need. We're getting an error here because we're trying to append two values to a data frame that has four values.

So for now I'm just going to put in N.A. placeholders for both of them. And actually for the number of shares to buy that N.A. placeholder is actually going to stick. So we only need to figure out the one year price return.

So we'll run this to get rid of that error and then look in here to see what we need to parse out for the one year price return. All right. Year one is what we need to search for.

Okay, here's the metric year one change percent. So we'll copy that. We will paste that there for now.

And then what we want to do is copy this. Paste it there and then add this in as an additional level of parsing to that. All right, let's run this.

(2:00:49 - 2:01:54)

I'm going to take this off and this means that instead of looping over just the first symbol string, we're going to loop over all of the symbol strings. And then inside of this big loop, I'm going to print the final data frame so we can see what it looks like. With any luck, this should be okay.

Oh, I'm sorry, I'm going to do this again and remove this print statement. All right, float object is not subscriptable. Hmm, that's because we need to change this endpoint to stats not price.

All right, so this will take a sec to run. It's a pretty big data frame to build. Awesome.

Everything looks good. So just to recap, what we did here was we created an empty pandas data frame with the columns that are specified here. And then we looped over all of the symbol strings in our symbol strings object.

We created a batch API call URL for those symbol strings that hits two different endpoints. It hits the price endpoint and it hits the stats endpoint. And then we used the requests library to execute an HTTP request to get that data in the form of a JSON object.

(2:01:54 - 2:02:26)

For every string in that comma separated string of string, or sorry, for every symbol in that comma separated string of symbols, we split them using the split method to get a list of them. And then we looped over that list to append the data for each symbol to our empty pandas data frame. And then we printed the pandas data frame and it looks like this.

Everything seems to be working so far so we can move on to removing low momentum stocks from our pandas data frame. All right. So like the other code cells in this project, this has a bit of background information.

(2:02:26 - 2:02:41)

It says the investment strategy that we're building seeks to identify the 50 highest momentum stocks in the S&P 500. Because of this, the next thing we need to do is remove all the stocks in our data frame that fall below this momentum threshold. To do that, we'll sort the pandas data frame by one year price return and then drop all stocks outside the top 50.

(2:02:42 - 2:03:07)

That sounds easy enough and pandas actually makes it very easy to sort data within the data frame. So we're going to use some built-in functionality of the pandas library to do this. All right.

So the first thing we're going to do is call final data frame and then on this final data frame, we're going to call the pandas method that allows us to sort the data frame based on values containing its columns and that method is sort values. Now this takes a few different parameters. The first one is the column that you want to use to source.

(2:03:07 - 2:03:16)

So we're going to use one year price return. If you just type in one and then tab, it might autocomplete. Now, if it doesn't autocomplete, you can just go up here and copy it.

(2:03:16 - 2:03:36)

Copying is always a good idea because there's lots of different ways you could spell this and just kind of avoid any typos. Although typos are a pretty easy bug to fix. All right.

So one year price return. And then the next parameter it accepts is whether or not you want it to sort ascending or sort it by descending. Now we want to sort descending values so that the highest momentum stocks are at the top.

(2:03:36 - 2:03:46)

So do that we'll say ascending equals false. And then the last thing we want to say is in place equals true. Now, if you've worked with pandas lots in the past, you're definitely familiar with this in place equals true method.

(2:03:47 - 2:04:08)

What this does is it changes the function from just returning a sorted data frame to actually modifying the original data frame. Now to see this in action, you can take it away, run this, and it shows the highest one year price return stocks at the top. But then if you run that line and then print the original data frame right below it, it will not return the sorted data frame.

(2:04:08 - 2:04:17)

It will just return the original one. So this shows you that without the in place equals true method, it doesn't actually modify the original data frame. Now, if you add the in place equals true method, that's not true.

(2:04:17 - 2:04:28)

It will return the sorted data frame. So that's how we sort the values. Now what we need to do is modify the pandas data frame so that it only includes the first 50 rows.

(2:04:28 - 2:04:45)

Now to do that, it's actually quite easy. You can just pass in the dash 50, yeah, dash 50, or sorry, not dash, colon 50, specification into square brackets like this. Now the indices are all different now because it's been sorted.

(2:04:45 - 2:04:50)

So I'm actually going to just make sure that this is right by doing length. Yeah, so it's 50. Great.

(2:04:50 - 2:05:06)

So this has 50 stocks now. And then to make this so that it actually modifies the original pandas data frame, all you need to do is say final data frame equals final data frame equals 50. And now if you print it, it will just return a data frame with 50 rows.

(2:05:07 - 2:05:22)

So awesome. So one last thing that we can do is to reset the index so it doesn't start and have

kind of this random list of numbers. So the easiest way to do that is just with pandas reset underscore index method doesn't actually accept any parameters.

(2:05:23 - 2:05:33)

But as you can see, it does the job. Now, just like before, without the in place equals true method, this doesn't actually modify the original data frame. So we'll have to specify that in here.

(2:05:36 - 2:05:55)

You'll notice as you work more and more with the pandas library that this in place equals true parameter works in almost all of the functions that you use regularly in the library. So there we go. So just to recap what we did here, we took our final data frame object, which contains price and momentum data on all the stocks in the S&P 500.

(2:05:56 - 2:06:17)

And we sorted its rows based on their one year price returns such that the highest momentum stocks are at the top. And then we use the in place equals true parameter to actually modify the original data frame instead of just returning a temporary copy. Once that was done in the next line, we modified the data frame such that it only contains the 50 stocks with the highest price momentum.

(2:06:17 - 2:06:31)

And then we use pandas reset underscore index method to change the index of the data frame so that it actually runs from 0 to 49 or 0 to 50 I should say, 0 to 49. Yeah, 0 to 49. And then we just printed out the final data frame on the last line.

(2:06:31 - 2:07:00)

So with that out of the way, we can move on to actually calculating the number of shares to modify for this simple momentum strategy. Alright, so just like in the last project, this is going to be kind of similar. The only change we're going to make is that we're going to wrap the functionality of accepting the portfolio size inside of a function because we'll actually be using the same functionality later in this tutorial when we build a better and more robust momentum strategy.

(2:07:00 - 2:07:07)

You can actually see that heading here. We started in the next code sale. So to do that, we're going to have to wrap it in a function and it's probably not necessary to wrap it in a function.

(2:07:07 - 2:07:20)

You could just copy-paste code. It is a Jupyter notebook after all, but it is definitely better to create functions if you're ever going to reuse any code. And since we're not building too many functions in this Python course, it's kind of a good excuse to practice building Python functions.

(2:07:20 - 2:07:38)

So if you're familiar with Python functions, and since we've already done this same functionality in the last project, try to pause this video here and try to complete the function yourself without watching me code through it. So break now and try that yourself. Alright, so what we're going to do is we're going to say def portfolio underscore size.

(2:07:39 - 2:07:54)

So if you're not familiar with function syntax in Python, this is just defining a new function called portfolio size. And the empty brackets here means that it accepts no parameters. And then the first thing we're going to do is we need to create a global variable in here.

(2:07:54 - 2:08:03)

So this function can define a variable and then it can be accessed outside that function. So the keyword to do that is global. And we're going to call this variable portfolio underscore size.

(2:08:03 - 2:08:33)

And then what we're going to do is we're going to call this portfolio portfolio underscore size equal to input. And then just like before, we're going to say enter this size of your portfolio. Now, as we saw in the last project, we need to make some special functionality here such that if someone tries to enter a string into this Python input, it will tell them to reenter a float value.

(2:08:34 - 2:08:58)

So what we're going to do is we're going to say try float portfolio underscore size. And if this returns an error, we're going to say print that is not a number. And then we're going to print again and say, please try again.

(2:09:01 - 2:09:16)

And then we are going to say basically this exact same functionality. Great. And then the only other change we need to make is to actually specialize this except statement so that it only works on the certain type of error that would happen if someone entered a string into this.

(2:09:16 - 2:09:23)

So that type of error is a value error. So we'll do that here. And actually, now that I look at this, this is a bit unnecessary with two separate print statements.

(2:09:23 - 2:09:31)

So I'm going to copy this here and do a new line character and try that. Okay. So let's run this to see if it defines properly.

(2:09:31 - 2:09:35)

Awesome. It does. So we're going to use this portfolio size function to accept a portfolio size now.

(2:09:35 - 2:09:49)

And then we're going to use it again later to accept a portfolio size for our better and more realistic momentum strategy. So now we're going to make sure this works. And actually, before we proceed, one thing that's worth knowing is that this function should be called portfolio input.

(2:09:49 - 2:10:04)

And it doesn't actually have to be called portfolio input, but it definitely should not be called portfolio size because then it has the same name as this variable. And that's kind of an easy way to introduce logical bugs into your code is by having a function and a variable with the same name. So we're going to change this to portfolio input, and then we're going to run this code cell to define it.

(2:10:04 - 2:10:19)

Then we're going to test it by running portfolio underscore input. And then we are going to print portfolio underscore size. Now here it's especially important to print the variable that's defined inside the function because we need to make sure that this global keyword is working properly.

(2:10:19 - 2:10:26)

So let's run this and see what happens. So another size of your portfolio will enter a thousand dollars and it should print a thousand dollars. Awesome.

(2:10:26 - 2:10:36)

Now let's try this again but put in a string. So we'll say my portfolio is too small to matter to you. That's kind of sad, but we're going to try it.

(2:10:36 - 2:10:38)

And then it says that is not a number. Please try again. Okay.

(2:10:38 - 2:10:43)

So my portfolio is one. Awesome. So our portfolio input function is working properly.

(2:10:43 - 2:10:57)

We can now move on to looping through our pandas data frame and calculating the number of shares to buy. So the first thing we need to do is actually calculate our position size and to do that I'm going to actually enter a slightly larger portfolio. So we're going to say that's a thousand.

(2:10:58 - 2:11:03)

That's a million. Let's do 10 million. And what we need to do is calculate position size.

(2:11:03 - 2:11:21)

So to calculate position size we're going to do position size is equal to float portfolio size divided by the length of our pandas data frame. So we'll say length of final data frame and then you can take the length of any column. I usually do dot index for this.

(2:11:21 - 2:11:33)

So this gives us let's print it out to see what the position size is. This gives us is that 200,000? Yep. Yep.

(2:11:33 - 2:11:38)

200,000. And that makes sense because we're just doing 10 million divided by 50. So that kind of math checks out.

(2:11:38 - 2:11:55)

And then the next thing to do is we need to loop through all of the rows in the pandas data frame. So for that we're going to say 4i in range, 0 and then the length of the data frame. Final data frame and then let's just print i to see what happens.

(2:11:56 - 2:12:07)

So this goes from 0 to 49. And if we go back up to our pandas data frame, we can see that the last index is 49 and the first index is 0. So that's awesome. That looks like it should work properly.

(2:12:08 - 2:12:21)

Let's take away this print statement now. And we're going to loop through all of the entries within the number of shares to buy column. Now the way to do that is with pandas loc functionality.

(2:12:21 - 2:12:37)

We're going to pass in i for the row and we're going to pass in number of shares to buy for the column. Now what we want to do here is just make sure that we're accessing this properly. So we're going to say 0 and then at the bottom we're going to say final underscore data frame.

(2:12:37 - 2:12:52)

Now if this works properly what should happen is every entry in the column of number of shares to buy should be replaced with a 0. So let's try this. Awesome. Now we're getting a warning here because it's saying I think this is actually a bug with pandas although I'm not sure of the setting with copy warning.

(2:12:52 - 2:13:03)

Because it says value is trying to be set on a copy of a slice from a data frame. So we're going to try using dot loc equals value instead and that's kind of exactly what we're doing. So I'm not really sure what's going on here.

(2:13:03 - 2:13:19)

But you can definitely ignore that for now. So now what we need to do is calculate actually how many shares to buy. So to do that we're going to have to access the price column of the data frame and we're going to have to divide the position size by that price.

(2:13:19 - 2:13:31)

So first we'll take position size and then we will divide it by this loc method. But instead of number of shares to buy we're just going to replace it with price. Now let's run this and see what happens.

(2:13:33 - 2:13:47)

Awesome so as you can see this is giving us fractional shares for everything and like we talked about in the last section of this course we don't want fractional shares for this. So the last thing we need to do is wrap all of this in a math dot floor function. Oops.

(2:13:48 - 2:13:57)

And this will round every number in this number of shares to buy column to the lowest closest energy. So let's try that. Awesome.

(2:13:57 - 2:14:06)

It's looking good. I think we are ready to proceed to building our better and more realistic momentum strategy. So let's do that now.

(2:14:07 - 2:14:17)

All right. So it's time for us to build a better and more realistic quantitative momentum strategy. Now what does it mean for a momentum strategy to be better? There's a lot of different ways you could define it.

(2:14:17 - 2:14:43)

Obviously a strategy is better if it performs better but it's kind of hard to tell that in without foresight. So what I mean by building a better momentum strategy is that real world quantitative investment firms typically differentiate between high quality momentum stocks and low quality momentum stocks. So a high quality momentum stock is a stock that over a long period of time shows slow and steady price returns.

(2:14:43 - 2:15:11)

So you know if a stock returns half a percent every month for a year that would be kind of like a high quality slow and steady momentum stock. A low quality momentum stock might not show any momentum for a long period of time and then surge upwards over a short period of time. So if you have a stock that returns zero percent for 11 months and then 10 percent in one month that would be considered a low quality momentum stock even though its one year price return would be 10 percent it might be better than a lot of other stocks.

(2:15:11 - 2:15:56)

So it might be hard to tell why that definition of high quality versus low quality momentum stocks is important but the reason why is because you want stocks that are seeing price returns in reaction to slow and steady business progress not one time business news. So the example that I used here is I said the reason why high quality momentum stocks are preferred is because low quality momentum can often be caused by short term news that is unlikely to be repeated in the future like an FDA drug approval for a biotechnology company. So this kind of begs the question how do you identify high quality momentum stocks and the best way to do it is to take a basket of different time series momentum metrics and use them all when calculating with momentum stocks to include in your strategy.

(2:15:56 - 2:16:21)

So to identify high quality momentum we're going to build a quantitative momentum strategy that selects stocks that have the highest percentiles of one month price return, three month price returns, six months price returns and one year price returns. And we're going to call this our high quality momentum strategy so you'll see that I use the abbreviation of HQM fairly often and that stands for high quality momentum. Let's start by building our data frame.

(2:16:22 - 2:16:41)

So the first thing we need to do is actually specify our columns. This is going to be a much longer Python list than what we used in the past for columns because it's going to include so

many more metrics. So the first thing we're going to use is ticker and then price and then number of shares to buy.

(2:16:45 - 2:17:59)

We're going to have to create a bunch of slots for our return metrics and our percentile metrics. So to give you like a broad overview we're going to fill in one column for each time period that has how much that stock has returned in the last year and then right beside that column there's going to be another column that has the percentile score of that return relative to all of the other stocks in the universe. So we're going to do one year price return, one year return percentile and then six month price return and then six month return percentile and then what's the next one three months three month price return and then three month return percentile and then one month price return and one month return percentile.

(2:17:59 - 2:18:16)

Alright, let's run this just to make sure it's formatted properly. Awesome. So now we can create our data frame which is going to be hqmdataframe is equal to pd.dataframe and we're going to specify columns equals hqm underscore columns.

(2:18:17 - 2:18:24)

Awesome. Now let's print this hqmdataframe to make sure that it is an empty pandas data frame with the columns that we specified. Awesome.

(2:18:24 - 2:18:49)

Now we can start looping over our simple string object that we created earlier for symbol string and symbol strings and here's where we have to create our batch API calls and do kind of all the similar logics that we used before. So the first thing we're going to do is execute our batch API call. So we can actually use very similar logic to that as before.

(2:18:50 - 2:19:12)

So we're going to scroll up here find our old batch API call and use the same logic because we don't actually need any metrics that weren't included in this. So we'll copy both of these lines so what we're grabbing here is the batch API call URL and the HTTP request that grabs the JSON object from that URL and stores it in our Python script. So we'll scroll back down to where we were here and we're going to add this to this loop.

(2:19:15 - 2:19:28)

All right. And now we need to loop through all of the symbols in that symbol string. So we're going to do for symbol in symbol underscore string dot split on the comma characters just like before.

(2:19:31 - 2:19:47)

And then in here we need to parse out all of these metrics. So whenever I'm doing high level parsing like this I find that it's easiest to just literally copy this in and then you can replace each metric with what you need. So this is going to be an append method first.

(2:19:47 - 2:20:01)

HQM underscore data frame is equal to HQM underscore data frame dot append and then in here we have to pass in all the things that we've been using throughout this course. So first it'll be a panda series and inside that panda series there'll be a python list. We'll fill that in later.

(2:20:02 - 2:20:12)

And then we have to say index equals HQM underscore columns. And then here we have to add the special ignore index equals true. All right.

(2:20:12 - 2:20:21)

Let's move back inside this python list. We're going to paste those metrics and then I'm just going to get the indentation looking a little better here. So I'll do that.

(2:20:22 - 2:20:41)

All right. So one easy thing we can do is all of the percentile metrics we're just going to fill in to start with N.A. And we will loop back through all of those metrics and use the SciPy library to calculate their percentile scores later. All right.

(2:20:41 - 2:20:53)

So that's easy. ticker will be replaced by symbol which is the incrementer of the for loop that we're in here. Price will be replaced by the price metric the price endpoint from the IAC cloud API.

(2:20:53 - 2:21:18)

So to do that we're going to create data and then symbol and then price. The number of shares to buy just like before will be filled in with an N.A. placeholder and the one year price return we actually already parsed out in our last example. So we can just go back up and copy that down to our new high quality momentum strategy.

(2:21:20 - 2:21:24)

All right. So here's the original parsing of the one year price return. We can copy that.

(2:21:25 - 2:21:33)

Scroll back down to where we were. Sorry right here. All right.

(2:21:33 - 2:21:46)

And we will replace this with the parsing of the equivalent metric. Awesome. And then as you can see here we are just going to copy this down to all of the other price returns and then change them to fit.

(2:21:46 - 2:22:04)

So this is month six instead of year one. And then we'll copy this and move it down to here and this will be month three instead of month six and this will be month one instead of month three. All right.

(2:22:05 - 2:22:15)

So let's run this to see if it works and then I'll do a quick recap of everything we did in that cell. So there is invalid syntax here. There's a missing comma there.

(2:22:17 - 2:22:35)

Let's try this again. Length of pass values is eleven index implies ten. What's going on? One two three four five six seven eight nine ten eleven.

(2:22:36 - 2:22:45)

So we were missing a comma here and if we run this now it should work. All right. So that is running correctly it looks like.

(2:22:46 - 2:22:55)

We're going to run this again and print out the final data. Sorry it's not the final data frame anymore. It's the HQM data frame and see what happens.

(2:23:01 - 2:23:18)

All right. So this is a big data frame quite a bit bigger than the ones we worked with in the past. It has ticker price number of shares to buy one year price return one year return percentile six month price return six month return percentile three month price return three month return percentile one month price return and one month return percentile.

(2:23:18 - 2:23:42)

So we have lots left to do to this data frame but first we're going to go through to each of these percentile columns and we're going to replace these N.A. values with a percentile score that shows how high a company's momentum is for that time period relative to all the other stocks in the universe. So this is where we're going to do this at. There's going to be a lot of repeated

logic in this section.

(2:23:42 - 2:24:33)

So I find the easiest way to do this is to create a ray called time periods and then inside this array we're going to create a string for each time period that we're going to be looping over and then we can use string interpolation to make this much easier. So we're basically going to say our time periods are one year and six month and three month and one month. Now if it's not exactly clear yet how we're going to be using this what we're going to do is we're going to loop through all of our columns and replace this value with a value from this list so that we have to write the logic ones but we can actually use a loop to calculate the percentile scores for all of the percentile columns.

(2:24:33 - 2:24:52)

So that's kind of what we're doing here. Now the next thing to do is create the actual loop. So for or row in HQM dot data frame dot index and then we need to use another loop inside of this.

(2:24:52 - 2:25:12)

This inner loop is going to loop over the time periods object that we just created. So for time period in time periods and then here we're going to use the pandas loc method to change the value of each time period column. So we're going to say HQM data frame dot loc.

(2:25:14 - 2:25:37)

The row is going to be this row incrementer that we're looping over here and then the column is going to be we're going to make an F string here and we're going to say it's going to look something like this with an interpolated value. So let's copy that. We will put that in here and then this is the time period that we want to interpolate.

(2:25:37 - 2:26:11)

So we will oh and just so you know you've probably seen me do this quite a few times in this course. All that I'm doing there is I highlight the string and then I do a you know I enter the curly bracket character. So I just enter that character and it will automatically wrap whatever you have highlighted in those curly brackets.

It's quite a nice little feature. Okay, so we're going to replace this with time period and then what we want to do is assign that to something. So to start let's just try assigning them all to zero and then oops didn't need to edit that and then at the end of this code cell we will print HQM underscore data frame.

(2:26:12 - 2:32:49)

Let's see what happens. All right, so you can see all the percentile scores are being replaced with zero which is awesome. But that's not quite exactly what we need to do yet.

We need to actually use the sci-fi stats module to calculate the percentile score based on whatever is inside of this. So let's do that now. The function we're going to use is stats dot percentile of score and then how this function works is it takes two arguments.

The first argument is the entire column and then the second argument is an entry from that column. So to start we're going to need the column which is simply HQM underscore data frame. I'm going to scroll this up to make sure you can see it.

And then we need to pass in this string. Sorry, not that string. We need to pass in this string as the column name and then interpolate in the time period.

So it will look like this. Awesome. Now what we need to do next is interpolate in the time period value.

So we need to change this to an F string and then change this to time period. Alright, the last thing we need to do is put a comma here and add in the actual value that we want to calculate. So what we're going to do is use the LLC method.

I'm going to copy this and put it in as a placeholder and then all I have to do is change the column name to match this. Now I know that this is a really long Python line. We're going to refactor it in a second.

So just hang tight. Copy that and put that in here. Alright, looks like I got some duplicate Fs in parentheses.

Okay, let's try this. Alright, so these look quite good. You can see that Apple, which has one of the highest percentile scores for one year, has a sorry, has one of the highest momentum scores for one year, has a 99th percentile.

So that's good. Everything here looks pretty logical to me. So what we're going to do now is refactor this code so it's much more readable.

So first thing we're going to do is we're going to say change column equals something and percentile column is equal to something. The percentile column is this. So we can copy that and replace this with this.

And the return column is this or the change column I mean, sorry. Alright, so let's go back through and replace these. And we'll go down here and replace this with the same thing.

Okay, this is quite a bit more readable. Scroll up so you can see what I actually did here. So basically, this is saying we're going to take the HTML data frame and at row of row and in the percentile column, we're going to assign it to this calculation stats dot percentile of score.

And then we're going to call this HTML data frame at change call. So this is the column that we're using to base our percentile calculations on and this is the entry from that column whose percentile score we want to calculate. Now, one other thing that we could do to this is we could say change this import somehow.

So if we go back to the top and we say, copy this and move up to here and from sci-fi import stats and then we could say, okay, percentile of score is what we want. Score as score. Okay, so what this is, if we run this import now, so we're saying from sci-fi's stats module, import the percentile of score function as score.

So we can go down and erase that very long function call and just type score instead. And all those changes will make this much more readable. It means you won't have to scroll horizontally to actually see the entire line.

Just so you know, in Python and most software development, it's kind of considered a bad practice to have really long lines. So if you can break it up like this, it's much more readable and much better. I think I believe too much.

Take this. All right. So that almost fits on one line now.

It's still quite a long call, but if we run this now, it should have all the same functionality. Great. So now we can move on to actually calculating the HQM score.

Now, we haven't actually talked about the HQM score yet, so let's do that now. The HQM score is just going to be an arithmetic mean of all the percentile scores for the momentum returns that we calculated in the last section. So to do that, we're going to import the mean function from Python's built-in statistics module.

So we're going to say from statistics, import mean. Let's run that to see if it works properly. Now, the way the mean function works is you can just say like a few numbers in here.

What's the mean of two and six? Well, this should be four and mean takes one position. Sorry, this has to be a list in here. Yeah.

So that's kind of how the mean function works. It's kind of exactly what you'd expect. We're going to use that to calculate the average of all the percentile scores that we calculated in the last section of this project.

And we actually have to do this row by row. So the first thing to do is to create a loop. We've been using lots of loops in here, but it's good practice.

So for row in HQM data frame dot index. And then the first thing we're going to do is create a empty python list called momentum percentiles. And then we're going to loop through all of the appropriate columns in the data frame to get the momentum score for that row.

So let's first do momentum percentiles equals empty python list. And then we need to loop

through all of the time periods in the time period object that we created earlier. So for time period in time periods.

(2:32:50 - 2:33:06)

And then we need to append the momentum scores to this momentum percentiles list. So momentum percentiles dot append. And then in here we want to say an loc method on the HTML data frame.

(2:33:06 - 2:34:26)

HQM underscore data frame dot loc. The first entry will be the row. The second entry will be an F string.

And inside that F string we are going to say time period and then return percentile. Let's try this just once and see if we actually get a reasonable momentum percentiles object. All right.

So we get eighty five, eighty five, sixty two, thirty five. Let's go up here and see if that looks right. Eighty five, eighty five, sixty two, and thirty five.

Awesome. So now what we can do is calculate the mean and we're going to put that mean in the HQM score column. Now looking at this in hindsight, we don't actually have an HQM score column, do we? What we're going to do now is go back up to the top and add our HQM score to our columns.

So first add this to this column. For now it's going to be an NA placeholder just like all of these were. So we have to add an NA placeholder here.

And now we just need to go back to here and rerun all of our code cells. If this runs correctly then we know that everything we just did worked. Awesome.

(2:34:27 - 2:34:54)

So now we need to say that HQM column needs to be assigned the mean of all of the values in the momentum percentiles list. So we can do that again using an LLC method. We'll say HQM data frame dot LLC and then the first entry of this is row, the second is HQM store, and this needs to be assigned the mean of momentum percentiles.

(2:34:55 - 2:35:59)

And now if we print the HQM data frame after running this, all of the entries in our HQM data frame should have a HQM score calculated on the very right. Now the only other change we need to make is to delete this limiter on the end of index and let's see how this looks. Awesome.

So it looks like we've successfully calculated an HQM score for every stock in our pandas data frame. Let's do a quick recap of what we did here. So from the Python statistics module we

imported the mean function and then we looped over all of the rows in our pandas data frame.

For each row we re-instantiated an empty Python list called momentum percentiles and then for every time period in our time periods list we loop through the time periods and to the momentum percentiles list we appended the percentile scores for that stock and then we used the LOC method to assign the average of all of those percentile scores to the HQM score column. Then to conclude we printed out that pandas data frame. All right, let's move on to selecting the 50 best momentum stocks.

(2:36:00 - 2:37:02)

So as before we can identify the 50 best momentum stocks in our universe by sorting the pandas data frame on the score that matters stuff. So in the last section that was one year price return and in this section it's actually going to be HQM score. So the logic here is very similar.

We're going to do HQM underscore data frame and we're going to use the sort values method. Sort underscore values. And what we're going to pass in here is the HQM score and then we also need ascending equals false.

So this will put our highest ranked momentum stocks at the top and the last thing we need is in place equals true. This will cause the sort values function to actually modify the original data frame. So let's print it out and see what this looks like.

All right, we can see our top few stocks all have extremely high momentum here which is a sign. Okay. The next thing we need to do is to drop all stocks except for the top 50.

(2:37:02 - 2:37:33)

And we saw how to do this earlier. You just say HQM data frame equals HQM data frame and then index it from colon 50. And if we print this out now you can see that it is shorter than it was before.

It's hard to tell how much because we haven't reset the index yet. So let's do reset underscore index. Sorry, that's a method not an attribute.

(2:37:34 - 2:37:53)

And it goes from 0 to 49. Awesome. The last thing we need to do is place in place equals true into this.

And then if we print it out we should see our finalized HQM data frame that goes from 0 to 49 on the index column. Awesome. Great.

(2:37:53 - 2:38:50)

So we are ready to start calculating the number of shares to buy and we'll use the portfolio

input function that we created earlier. So let's run that. And name portfolio input is not defined.

That's because I spelt it wrong. Let's do a $1 million portfolio. And then in the next section we need to calculate the number of shares to buy.

And this is exactly what we did before. So first we need to calculate position size which is going to be the size of our portfolio divided by how many stocks are in it. So the first thing we need to do is take the float value of portfolio underscore size and then divide it by the length of our HQM data frame.

And you can take any column here. I usually take the index just like that. And then let's print out position size.

20,000 that looks right. Awesome. Now we need to loop through a row of our pandas data frame.

(2:38:50 - 2:39:26)

So we'll say 4i in HQM data frame dot index. And then we need to assign the number of shares to buy to the number of shares to buy columns. So we'll use the LOC method to do that HQM data frame dot LOC.

And then we'll pass in i comma and then the number of shares to buy. To test this out let's assign them all a value of zero first and then print it out to see what happens. That comma shouldn't be there.

(2:39:27 - 2:40:51)

Awesome. Number of shares to buy has all been assigned zero. Great.

So like before, what we're going to do now is we're going to divide our position size by the company's stock price. And the easiest way to do that is to copy this over and replace the number of shares to buy column with the price column. Price.

Okay, this looks good. We're getting decimals again. So the last thing to do is wrap this in the math dot for function.

And just like that, we successfully calculated the number of shares to buy for each stock in our HQM data frame. Oh, there's one thing actually that we did wrong earlier. So you know just now that there's a new column called index.

And the reason why that's there is because when we did our reset index earlier, I can't believe I didn't notice this at the time. But there's an optional parameter here. And just to give you a bit more information about how to use Jupyter Notebooks, if you want to know what possible parameters are available for a function or a method, you can do shift tab and this will pop up.

And what we want to do is say drop equals true. What this does is drop equals true. What this does is the old index just gets removed from the pandas data frame completely.

That's not the case. I'm going to show you exactly how you debug this. I'm going to do pandas dot and then reset index to go to the actual documentation.

(2:41:28 - 2:43:12)

Oh, sorry. I'll tell you exactly what the bug is here. The reason why this happened is because we ran this more than once.

So if we just go back up here to where we initialize the HQM data frame and then run all the way down, this problem will be solved. See, just like that it's gone. It goes index and then ticker.

Okay. So just to recap, we should have originally put this here. Drop equals true in our reset index method.

We didn't and that caused there to be an extra index column, which we've now fixed. So now when we run this, we get the number of shares to buy and all the other columns we need. All right.

So the next thing we need to do is format our pandas data frame into a Microsoft Excel document for use by non-technical OSI users. So the first thing we need to do to do this, we're going to actually back up a bit. To do this, we're going to be using the same library as we did in the last project, which is XLSX writer.

And to start, the first thing we need to do is initialize a writer object. So to do that, you do writer equals kd.xcel writer. And then the first thing you need to pass in is the name of the Excel file that you want to create.

In this case, we're going to call it momentum underscore strategy.xlsx. The next thing you want to add is the engine, which is engine equals xlsx writer. All right. So the next thing we need to do is say H2M data frame dot two underscore Excel.

And the first argument of this function is going to be the writer object we just created. The second argument is going to be the sheet name. In this case, it's going to be sheet underscore name equals momentum strategy.

(2:43:14 - 2:45:18)

And the last argument is going to be index equals false. All right. So run that code cell and we can proceed.

Now, the next thing we need to do is actually create the formats that we need. We actually created all of these formats in the last section of this course. So or in the last project of this course, I should say.

So I'm not going to make you work through this again. Instead, we're just going to run this code cell to proceed. And now we need to actually loop through the columns format dictionary again to apply the formats that we created to the columns that we have.

So the easiest way to do this, in my opinion, is to scroll all the way back here to where we initially created our columns, which is right here. And then I'm going to copy all of these and use this as kind of a basis for our dictionary that we're going to build. All right.

So we're going to paste that there. Now we need to start with column name. So it's just going to, we're just going to be doing this to start A, B, C, D, E. Now note that if I was in like an actual development environment, I typically write big programs in the VS code.

I can do this much faster with multi cursor. Jupyter notebooks are better for visualizations and other sorts of things, though. All right.

So our columns are going to be from A through L. And then we're going to want to wrap each of these inside of a list. Now, depending on your operating system, there's different ways that you can enable this multi cursor functionality that I'm using right now. I am using Linux and I'm just control clicking so that I can add the start of the list to each line.

(2:45:19 - 2:46:39)

And then, oops, I'm going to move back to the start of the list. Oh, shoot. Try this again.

Now what I'm going to do is try to move to the end of the line and add the closing list brackets. Not going to be super pretty because some of these have columns and some of them don't. Move back before the karma.

Try that. Okay. What do we have? Any of these broken? The last one is great.

Okay. So now we have a columns format dictionary where the keys are each column letter and the values are a list with one item where that one item is the name of that column. The last thing we need to do is work through these one by one and apply the template we want.

So this will be a string underscore template. Price will be dollar underscore template. Number of shares to buy will be string integer template not string template.

One year price return will be percent template. One year return percentile. So most of these are going to be percent template moving down.

So I'm just going to copy this and paste it. All right. This is looking good so far.

(2:46:39 - 2:47:05)

Let's run this code cell next to make sure that this dictionary is actually formatted properly. So it ran correctly, which is good. All right.

So the next thing we need to do is move through every key in this dictionary and apply the right formats to that column. So let's try this. We're going to say for column in column underscore formats dot keys and let's just print column and make sure this is working right.

(2:47:05 - 2:47:40)

A B C D E F G H I J K L. That looks good. All right. The next thing we need to do is use the method for Excel as a writer that actually applies a format to a specific column.

As we saw in the last project, the way to do that is we need to call the writer object and then from that writer object we need to access its sheets attribute, which is a list of all the sheets in the Excel file. And then we need to access the momentum strategy tab from that list. And then we need to call the set column method on that.

(2:47:40 - 2:48:17)

I was going to take three parameters. So let's put two commas in here to reflect that. The first parameter is the format that we want to apply.

So the first parameter is the column that we want to apply the format to. So it will look something like this A through A. The second parameter is the column width, which we're going to say is 18 pixels. Actually, let's do something like 22.

And then the last is which column we're going to want to apply to that. So we'll say string underscore format. And then let's just test that this is working.

I will say writer dot save. And then if we run this, what happens writer dot sheets and momentum strategy string format is not. Oh, it's because string template.

(2:48:18 - 2:48:34)

All right. Now let's go to our folder and see if this is working right. Starter files, momentum strategy.

So it's there, which is good. If we open this up, we can see that the get out of here. We can see that the first column has been formatted properly.

(2:48:34 - 2:49:22)

We're going to go format the headers in the same way that we did in the last section. So don't worry about that yet. All right.

So now we need to change this set column method such that it actually is dynamically applying formats to each column in the deadline. So first thing to do is change the column like this. And the second thing to do is to add a to here, we're going to access the column format dictionary.

We're going to pass in the column key. All right. Let's try that.

What happens now columns is not defined. That's because that should be column. Now if we go to our Excel file, it's still not working.

And that is we saw this in the last project too. We need to go back up and reinitialize our writer object and go down to the bottom again. All right.

(2:49:22 - 2:50:51)

What is happening here? We have some kind of bug on writer dot save. Why is that save? Oh, I see. Okay.

So here in the writer dot sheets dot set column method, this is actually a list and we need to actually parse it to get the format that we want. That format is in the second item of the list, which is that index one. So if we pass one there, then this should work properly and we can go see our, oops, ignore that.

We can go see our newly formatted Excel file. All right. So the formats are being applied to each column properly.

You can see that they're not actually wide enough because the column names in this project are longer than the column names in the last project. So let's change that 22 to 25. And then the last thing we need to do is actually format the column headers.

And we saw in the last section that the easiest way to do that is just overriding them using the right methods. So the right method is similar to the set column method, except instead of taking an entire column for the first one, it takes just a cell from a column. So what we're going to do here is we're going to access the column and then one.

So that's the first cell of that column. Here we're going to specify the column width. And then here we are going to do the exact same thing and apply the format from the second entry of that list.

(2:50:51 - 2:51:14)

All right. Let's go back to the top of our formatting the Excel format section, run through all of this again. And with any luck, this should be formatted exactly like we want.

There we go. All right. So you can see that this is not good because what is going on in this top row? Well, oh, I see.

(2:51:14 - 2:52:08)

This doesn't actually take column width. This takes the value that we want to put in it. And that's just a mistake on my part, a logical mistake.

That should be this list at index zero. So let's copy that list, put it here and put index zero. We'll

move this writer dot save method back down to the last column of this Jupiter notebook or the last cell of this Jupiter notebook rather then we'll move back up to where the writer object was initialized and run it to the bottom.

Now when we open up our Momentum Strategy Excel file, we should see that the entire notebook is done. We have ticker, price, number of shares to buy, one year price return, one year return percentile, six month price return, six month return percentile, three month price return, three month return percentile, one month price return, one month return percentile and HQM score. Now you can see that all of these aren't actually formatted like they should be.

(2:52:08 - 2:52:56)

So all these percentile scores are multiplied by I guess 100 over what their actual value should be. So let's go back into our notebook and fix that. If we scroll down to the bottom, we're going to scroll up here.

We're going to go all the way back to where our percentile scores were originally calculated. And that is right here, I think. Yeah, so no, sorry, this is the HQM score.

We need to go up a little bit further. Right. So this is where they originally calculated.

We're going to move to the end of this calculation and divide it by 100. And then we're going to go back to where we created our HQM data frame and run this entire notebook to the very end. All right.

(2:52:56 - 2:53:49)

So this should all run with no problems. We were waiting on our input here so I'll do a million dollars again. And this should run to the bottom with no problem.

All right. Let's check out our new Excel file to see if those percentile scores were reduced in magnitude like they should have been. All right.

So we have one year price return and 99.2. That looks good. All right. So you can see that this has been sorted by HQM score properly.

We go from 99.1 down to 81.9. And just like that we have a list of the 50 highest momentum stocks in the S&P 500 along with the number of shares that you should buy if you want to create an equal weight basket on those 50 stocks. So that concludes our second project for this algorithmic trading and Python course. I hope you enjoyed this project.

(2:53:49 - 2:54:38)

In the next section we're going to learn how to build a value investing strategy that buys the 50 cheapest stocks in the S&P 500 based on a composite score of a number of different value investing metrics. So I will see you there in the next project which is going to be a value

investing strategy. Now for anyone who's not familiar with value investing it just means trying to buy stocks and other investments that are cheap relative to our cell.

For anyone who's not familiar with value investing it just means trying to buy stocks and other investments that are cheap relative to our assessment of their intrinsic value. There's many different ways to try to estimate the intrinsic value of the stock or other investments. So what we're going to do in this project is we're going to build a simple value strategy that relies on one metric and then we're going to improve that value strategy by creating a composite value strategy that takes into account many different metrics when trying to estimate intrinsic value.

(2:54:39 - 2:55:50)

So to start let's navigate into I'm in my terminal here in my home directory. Let's navigate into our algorithmic trading and Python folder and then let's launch our Jupiter notebook. Sorry before we do that we actually have to launch our virtual environment and then we can launch our do the notebook with Python dash M notebook a Jupiter notebook will open up in your browser and then you're going to want to navigate into our starter files folder and open up the quantitative value strategy dot IPANB notebook.

All right. So like I said before value investing means investing in the stocks that are cheapest relative to common measures of business value like earnings or assets. For this project we're going to build an investing strategy that selects the 50 stocks with the best value metrics.

From there we will calculate recommended trades for an equal weight portfolio of these 50 stocks. So like before the first thing that we need to do is import the open source Python libraries that we'll be using through at our Jupiter notebook. So it's going to be the same libraries as we used in the last section so import NumPy as MP as a quick recap NumPy is a numerical computing library that you need to work with pandas and then we'll need to import pandas as PD then we'll need to import XLSX writer.

(2:55:51 - 2:56:42)

This is the module that we use to save our pandas data frame as an Excel file at the end of this Jupiter notebook. We're going to import the requests library which is used for making HTTP requests. You'll recall from our first project that the slogan for requests is HTTP for humans which I quite like and then we're also going to need to import the stats module of SciPy.

And the last thing we're going to import is the math module. So let's run this code cell to import all of these libraries into our Python script. Great, that's done.

The next thing we need to do is import our list of stocks and our API tokens. So just like before we're going to import our list of stocks into a pandas data frame called stocks and we're also going to import our IAS cloud token from our secret spot. So let's do that.

(2:56:42 - 3:03:22)

First we'll say stocks equals PD dot read underscore CSV and then we're going to type in SP and hit tab to auto complete the file name. Let's just print this to make sure that it is working like we intend. So this is a list of all the stocks and this is a pandas data frame.

You can get the pandas series that's associated with the only column that data frame by parsing out the ticker column. We'll do that later. And the other thing we need to do here is say from secrets import and then if you just hit tab here it'll give you some options.

What we want is this IAS cloud API token. Great. So that is how we import our list of stocks and API tokens.

And the next session we're going to make our first API call using our IAS cloud token. Great. So like before what we need to do now is figure out which endpoint of the IAS cloud API that we need to hit.

So I said in the introduction to this project that we're going to start by building a simple value strategy that screens for stocks based on a single value metric. And that value metric is going to be the price to earnings ratio. Now for anyone who's not familiar with a company's price earnings ratio it's basically you take their stock price and you divide it by how much earnings you expect them to have over the next year.

So as a basic example let's say we'll look at Apple again. So Apple's current stock price is five hundred and eight dollars and six cents. Now if you wanted to calculate a price to earnings ratio for Apple we need to find an earnings estimate.

So we'll just type Apple earnings estimate. And it looks like these are all quarterly estimates and we want an annual estimate. So whenever you're calculating a price earnings ratio or any other kind of valuation metric you always use annual numbers in the denominator.

So it looks like this chart from NASDAQ provides some estimated earnings metrics. So yearly earnings forecast. This is what we want.

And we will use the fiscal year ending in September 2021. It looks like they have consensus earnings per share of sixteen dollars per share. And that means that to calculate the price to earnings ratio we just calculate to take the company's current stock price and divide it by sixteen.

So what does this give us? Thirty one. So that means Apple's price earnings ratio is thirty one. That's some high level background on what the price earnings ratio is.

Fortunately IEX cloud makes it very easy to access price earnings ratios because if you go to their documentation they actually provide the price to earnings ratio natively here. So we'll just start by trying to control F find a price to earnings and it looks like we have it right here. So PE ratio in this API endpoint refers to the price to earnings ratio.

Oh I don't know what that was a link in here somewhere. I clicked the epic timestamp there. So anyways PE ratio refers to the price to earnings ratio for the company.

You know what endpoint is this? This is the quote endpoint. Now this is good because one of the other things that we'll need for this screener is we're going to need the latest stock price for the company. So you look in the JSON response here you'll notice that they have latest price here and they also have down at the bottom PE ratio.

So what we're going to do is we're going to take this API endpoint. We're going to copy it into our Jupyter Notebook and we are going to go back to the sandbox mode here and get our base URL for every API call that we're going to make. So here is the sandbox base URL.

We're going to copy that move it back here put it there and you'll recall from the first project that we actually have to add stable here to show that stable there to show that we want the latest stable version of the API and not any kind of bleeding edge version of the API that might have beta features that aren't fully tested yet. So this is going to be kind of a sample API endpoint that we'll use to get our price earnings ratio. We're going to call this API URL and we're going to make it a string.

Now we need to make two small changes to this. First of all we're going to take this field off the end because we don't actually want to parse a single metric out of this endpoint. We want to parse the entire Json response.

We're going to leave symbol there, but we're going to turn this into an F string so that that symbol actually becomes interpolated. Now we need to make a symbol variable here and we're going to say the symbol is apple and the next thing we need to do is create an actual API call using the requests library. So to do that we're going to say data equals requests dot get and then inside of this get method we're going to pass in this API underscore URL and then we are going to print out the status code of this HTTP request.

Now we've talked about this before, but status codes tell you whether your HTTP request was successful or not. A successful HTTP request will return a status code of 200 and most erroneous HTTP request will return a status code that's somewhere in the 400 range. So 500 is actually also an indicator error.

So we're going to do a print statement that prints out data dot status underscore code and if this gives us 200 then we know that our first API call was executed correctly. All right, so this gives us 400 and the reason why I've done this actually before in this course, but I forgot to add on our authentication token. So to do that we're just going to interpolate in this question mark token equals and then inside these curly brackets we're going to add our IEX Cloud API token and this gives us a status code of 200.

Awesome. Now we'll transform this data variable into a JSON object using the JSON method which accepts no parameters and then we can print out this JSON response to see what it

actually contains. Okay, so you can see that it has a number of different data points available in this HTTP response including symbol, company name, primary exchange, calculation price, whole bunch of other stuff.

You can see actually since we're using sandbox mode this primary exchange, quadons, that's not a real stock exchange. It looks like they actually just took the company's real exchange which is NASDAQ, spelled like this, NAS DAQ, and then they scrambled it or yeah, it looks like they scrambled it somehow. Anyways, that's kind of an interesting part of using the sandbox mode of the IEX Cloud API.

So what we want out of here is this latest price endpoint and this PE ratio endpoint here. So we'll parse out both of those variables in the next code cell. Okay, so the first thing we want to do is calculate it with the stock price.

(3:03:22 - 3:03:47)

So we'll say price equals data, and then this is just a Python dictionary. You can actually verify that by just printing out its type like this. This will give us a dict.

Yeah, it's a class of dict. So we'll take out that print statement. Since it's a Python dictionary, you can access the value stored in the dictionary by passing the keys and using square brackets.

(3:03:47 - 3:05:01)

We've seen this earlier in this course, but it's always good to have a little refresher. So the two things we want are so we will assign that to the price variable and let's just print it to make sure it is parsed out correctly 513.89. That's awesome. The other thing we want is price to earnings ratio.

So we'll say PE underscore ratio equals data. And then we're going to parse out this PE ratio string here. And just like we did with price, we're going to print this out to make sure that it is working properly.

PE ratio 39.6. You can see that this is a little bit higher than the 31 PE that we calculated manually. And that's probably because IEX Cloud is using a different kind of earnings. One of the tricky things about running any kind of value strategy quantitatively like this is that when you're using earnings or any kind of other fundamental metric, there are numerous different ways that they can be calculated.

You can use earnings estimates, which is what I did in my example. You can also use trailing earnings, which some people believe are more reliable because they're actually reported earnings instead of relying on someone's estimate. But if you ever calculate a PE ratio and you're confident that your math is right, but it doesn't match someone else's math, the reason why is probably because they're defining the ratio slightly differently.

(3:05:01 - 3:05:17)

So when you're doing any kind of work like this in the future, it's always good practice to strictly define what you're doing. So if you're using trailing earnings, say I'm using trailing earnings when I'm calculating my PE ratio. If you're using forward earnings, you might want to call it a forward PE ratio instead of just PE ratio so that it removes that ambiguity.

(3:05:18 - 3:05:44)

Alright, so like before, we can now move on to executing a batch API call. And just like I did in the momentum strategy for this course, I've actually provided some reusable code that we have used earlier on so that you don't actually have to kind of redundantly write the same code over and over again. Although if you're up for the practice, what you could do is just delete this entire code cell and try to write our chunks function and our symbol strings variable from scratch.

(3:05:44 - 3:06:57)

So for now, for the sake of brevity, I'm going to just run this code cell and proceed with executing our batch API call. So actually before I do that, I'll give you a quick recap of what's going on here. So first off, this comment at the top just tells you where I sourced this chunks function from.

Since I didn't write this myself, it's always good to attribute a function to wherever you received if it's someone else's work. So what does this chunks function actually do? This chunks function takes in a list, which is this LST, and it also takes in this variable n. And what it does is it, you can see in the doc string here that it yields successive and sized chunks from lists. And to be more specific, if you have a list of five hundred and five, like this, which is what we're going to pass in, it will return five lists of, and sorry, you pass in a list of five hundred and five and a value of n of one hundred.

It will pass you back a list that is five entries of length one hundred and one entry of length five. So that's what that is. Next up, we create a list using this chunks variable that takes all of our stocks and divides them into five lists of length one hundred and one length of list five.

(3:06:57 - 3:08:07)

And then we create an empty symbol strings list. And then with this for loop, we take every group of stocks that's in the symbol groups variable and we create a comma separated string that has all of the tickers from each list. And if you want to see what those look like, we can actually uncomment this print statement and run this code cell.

So you can see, this is one list. This is another list. This is the next list.

This is the end of the fourth. This is the end of the fifth and this is the entire fifth list, which is

shorter because it wasn't long enough to actually fill out the length of one hundred that was specified by the function. Okay.

And then the last thing that's done in this code cell is we specify the columns for our pandas data frame. So the columns are ticker, price, price earnings ratio and number of shares to buy. So I'm going to run this again with this print statement comment today to remove that large print.

Okay. We can now move on to writing code from scratch. So the first thing that we want to do is to create a blank pandas data frame.

And we saw how to do that. We're going to call it final data frame and we're going to assign it a value of PD dot data frame. And within this class instantiation, we're going to pass in columns equals my underscore columns.

(3:08:07 - 3:09:28)

Now we saw before that we could have just called this columns, but then you have columns equals columns and that can be a bit unreadable. So this is what we're going to do to make sure that that data frame was created properly. We're going to print it out.

And as you can see, it is an empty pandas data frame with four columns. And the column names are the four names that we specified in this list in the previous code cell. So now that that's done, what we need to do is we need to loop over every symbol string in our symbol strings object.

And to do that, we're going to say four symbol string in symbol strings. And then we need to actually create another loop inside of here that loops over every ticker in that symbol string. So we're going to say four symbol in symbol underscore string got split.

And you'll recall from the last two projects that if we call this split method, what it does is it basically does the opposite of this join method that we saw in the last code cell. So this is what our loop is going to be. And then we're just going to print out symbol.

And this will allow us to make sure that we're actually iterating over every symbol in our universe. Great. So it goes all the way from A and this should go to Z because these are in order.

Yep, all the way down to Z. So that looks like it's working okay. All right. So within the first four loop but outside of the second four loop, we're going to create a variable.

(3:09:29 - 3:11:32)

We're going to call that variable batch API call URL. And what is this going to be? It's going to be a URL that is a string and it's going to be an F string and it's going to query this endpoint that we used earlier, this quote endpoint for every stock in our universe. And it's going to do that in

batch API calls for a hundred stocks at a time.

Now to get the proper syntax for that, we're going to go back to the IEX Cloud docs and we're going to search for batch API call. And that doesn't come up with anything. So I'm just going to search for batch and here you can see batch requests.

Now I'm going to scroll down here to this example, which is what we copied in the past. I'm going to copy the link address. I'm going to go back to my Jupiter notebook and I'm going to paste that in here.

And then we have to make a few modifications. So the first thing we want to change is the symbols. And what we're going to do here is we're going to actually change them into an interpolated value.

This interpolated value should be symbol underscore string. Now, what is that symbol string variable? It is the iterator of this first for loop. So it will be if you uncomment this and run this.

This is one example of such a symbol string that you can interpolate into that API call URL. So we'll comment that out and run it again. Great.

So there's a few other changes that we have to make to this batch API call URL. One of the first things is that if you look at the types of API endpoints that we're hitting here, we have quote, we have news and we have chart. We actually only need the quote endpoints.

So let's take away all the other ones. Awesome. And then there's two other parameters specified here, range and last.

And we don't need either of those. So we can take them both off and replace them with our token equals IEX five URL. All right.

So let's just run this to make sure that that's formatted properly. And instead of this print statement, we're going to comment it out and just replace it with a pass keyword. Oops, ignore that.

(3:11:37 - 3:13:24)

All right. So we're going to replace this with a pass keyword. Great.

So that ran properly, which indicates that the syntax is okay. Now it's time for us to actually, so we've created our API call URL, but we haven't actually executed the HTTP request yet. And that's what we'll do next.

So to do that, we're going to use the requests library to execute a get request. And we're going to store that response in a variable called data. So data equals requests, press dot get.

And we're going to pass in batch API call URL. And then we're going to call the JSON method on

that. Actually, no, we're just going to leave it as is and we're going to print data dot status underscore code for each one.

Now, since there's six entries in our list, if everything is working properly, then they should print 200 six times. We get 403 for each one. Now, this is actually interesting.

So I mentioned earlier that 400 and 500 codes and when I say 400 and 500 codes, I don't mean exactly 400 and exactly 500. I mean anything from 400 to 499 and anything from 500 to 599. These are generally indicate errors.

So let's go look exactly what type of error this is. I'm just going to type HTTP 403, run a search and Wikipedia is actually pretty good resource for this. It says the HTTP 403 is an HTTP status code, meaning access to the requested resources forbidden for some reason.

Now, this is a very, very specific HTTP response that usually means that you have you not entering the proper API token and I'll show you exactly why that is. I know without looking because we didn't change this cloud keyword to sandbox. If we do that again and run it, we should get six 200 codes.

(3:13:25 - 3:19:00)

Awesome. 123456. Great.

So the next thing we need to do is actually parse the data and using an append method, add it to the final data frame. So we've seen this before. What we need to do is add that here.

We'll take away this print statement and pass keyword and then we'll add the final underscore data frame equals final underscore data frame dot append. And then within this append method, we need to create a panda series. We've seen this before and within that panda series, we need to create a Python list.

So what goes inside this Python list? We need basically one data point for every column. We need ticker price price earnings ratio and the number of shares to buy. So we'll say symbol.

That's our ticker. We'll say data and then we'll parse out the latest price data point. And then we'll say data and we'll parse out the P.E. ratio data point.

And then for the number of shares to buy, we will parse out nothing and just say n under n slash a. Now for this panda series, we have to specify what the index is. We'll say index equals my columns. This tells the append method which columns of the pandas data frame need to be considered when you're appending it each time.

So that's very important. And then we also need to add ignore underscore index equals true. All right.

So if we run this code cell now and then print out our final data frame, we should get a data

frame that has 505 rows and each row corresponds to one stock with the four metrics that we selected. So with any luck, it should run successfully and we're getting response object is not subscriptable. I'll show you what the problem is here.

When we ran our HTTP request, we didn't do dot JSON on the end to actually turn it into a Python dictionary. So if we run this now, that should be fixed. Latest price.

Oh, I grossly oversimplified the parsing here. So since this is a batch API call, we actually need to do multiple levels of parsing. The first is symbol.

The second is which endpoint you want to hit. So if we add that to both of these parses, we should be okay. All right, so here's our data frame.

It goes from index zero to index five hundred and four, which indicates that all five hundred and five stocks have been successfully added, and it has four columns, ticker price, price to earnings ratio, and the number of shares to buy. So now that that's done, we can move on to removing glamour stocks. All right.

So now we need to remove the glamour stocks from our pandas data frame. And for anyone who's not familiar with the term glamour stock, it's basically the term that's used in the data frame used to describe the opposite of a value stock. So if a stock ranks very poorly on value metrics, then instead of being called a non-value stock or something like that, it's just called a glamour stock.

Now, since the goal of this strategy is to identify the 50 best value stocks from our universe, our next step is to remove glamour stocks from the data frame. And to do this, we'll kind of follow similar logic as we did before. We're going to sort the pandas data frame such that the stocks with the lowest price to earnings ratios are at the top, and then we're going to remove all of the entries from the data frame, except for stocks that fall below the top 50 in that ranking scheme.

So let's do that. The first thing we need to do is sort the pandas data frame. And through that, we will say final data frame dot sort underscore values.

Now, you'll recall that this is the exact same method applied to the pandas data frame that we use to sort values in the momentum strategy section of this course. And then what we want to pass in first is the column that we want to sort out, and that is price to earnings ratio. I just tried to do a tab auto complete and it didn't work.

So what I'm going to do is just go up here and copy this. That means I don't have to worry about making sure I get the capitalization right and everything. The next thing we need to do is specify the ascending parameter.

Now we want it to be descending, which means that sending is equal to false. And the last thing we need to say is in place equals true. This parameter tells pandas that instead of just returning a modified data frame, we want to modify the original data frame.

Let's print out the final data frame object to see if this worked correctly. So as you can see, we have a price earnings ratio of 2400 at the top and a price earnings ratio of not at the bottom. I actually did this backward.

This should be true. Now we see that our lowest values are at the top and our highest values are at the bottom. Now, one important thing to know about this is that since ascending defaults, having a true value, we can actually remove this completely and it'll give us the same output as you see.

Another thing that's worth mentioning is that all of these stocks that have negative price to earnings ratios, if you think about the mathematical definition of the price earnings ratio, it's price divided by earnings. So in order for price to earnings ratio to have a negative value, then either price has to be negative or earnings has to be negative. Now, stocks can't actually have a negative price.

So what this means is that these stocks have negative earnings. There is kind of a wide body of evidence to say that you might not want to invest in stocks that have negative earnings. So in a real investment strategy, you may want to remove all stocks that have negative price to earnings ratios.

And one easy way to do that would be something like this. Final data frame such that final data frame at the price to earnings ratio column is greater than zero. Now, what does this return? Should return the final data frame with all of the negative values removed.

(3:19:01 - 3:20:32)

So we're going to include this and to actually modify the original data frame, we have to assign it. You'll notice that if we print this, this gives us the final data frame with negative values. So what we're going to do is take this out.

We're going to say final data frame is equal to that. And then we need to return just the top 50 stocks. According to this order schema, we saw how to do that in the momentum project of this course.

It's pretty easy to say final data frame is equal to final underscore data frame. And then you want to access out with square brackets colon 50. Now, if we print this, it should be a data frame that ranges from zero to 50.

Okay, so this looks good to me. There's a couple of things we can do to make sure that this is working properly. We can print its length.

This should be 50. Awesome. And then the last thing we want to do is reset its index so that the index actually runs from zero to 50.

So we'll do reset underscore index. What does this give us? All right. As you can see, this now

goes from zero to 49, which means there's 50 entries in the pandas data frame.

The only other thing we need to do is get rid of this new index column. We saw one way to do it in the last section of this course. I'm going to show you a different way now.

First, one thing I forgot here is we want to say in place equals true within this. And that means that the reset index method will actually modify the original data frame. Now, if we print this out again, we should see the modified data frame.

(3:20:32 - 3:21:54)

Awesome. And since I ran this twice, we actually have two separate reset indices. So let's go back up here and we'll rerun this.

We run this and we run this and we should just have one duplicate index. Okay. What we want to do now is we're going to call the drop method and we're going to pass in the index column.

We're also going to pass in in place equals true. Here, what does this give us? It gives us a key error because index is not found in access. Okay.

Uh, the drop method by definition automatically acts on rows, not on columns. So to change that, we have to say access equals one. What does this give us? Cannot insert level underscore zero.

That's because we already ran that cell. So if we run this now, what happens? All right. Everything looks to appears to have worked properly.

So we'll print this out again to make sure it looks good. Awesome. So now let's just do a quick recap of what we learned in this section.

Um, I've explained briefly what a glamor stock is. Uh, we sorted our final data frame such that the stocks with the lowest price to earnings ratios were at the top of the data frame and we specified the in place equals true parameters so that the original data frame was actually modified and it didn't just return a modified data frame. So that's what that line does.

(3:21:55 - 3:25:12)

Next up we, uh, we sorted the data frames such that all rows that have a price to earnings ratio value below zero were dropped from the data frame. Then we trim the data frame to only include the stocks with the 50 lowest price to earnings ratios. We reset the index and then we drop the new index column that was created by way of this reset index method.

Now one important thing to note about this is that we could have actually specified the drop parameter here to do the same thing. That's what we did in the last section. But the reason why I showed you this method instead is just to reinforce the idea that whenever you're doing any kind of software development, there's usually multiple different ways to solve a given problem.

So that's that. Let's now move on to calculating the number of shares that we need to buy. Just like we did in the momentum strategy section of this course, we're going to use the portfolio input function to do this and to make things easy for you.

I actually included the portfolio input function there. I encourage you to try to delete that formula and recreate it from scratch if you want the extra practice. For now, I'm just going to run this and then I'm going to use the portfolio input function to accept a portfolio size variable.

So let's do that. And I'm going to test here that the try accept value error statement is working by putting in a string. So I'm going to say my portfolio is one million dollars.

And then if I enter this, then it should print it. That's not a number. Try again.

Awesome. Now I can say one million dollars. That's six zeros, one, two, three, four, five, six.

Awesome. The next thing we need to do is calculate the position size. So just like we did before, position size can be calculated by dividing the portfolio size by the number of shares in the portfolio.

So we're in the position size divided by the length of our final data frame. And you can take any column of this data frame to find its length. I'm going to use the index and there's no kind of right or wrong answer here.

I just, the index is what I typically use for this. So that's that. And this position size should be twenty thousand dollars, I believe.

Now here it says type error unsupported operand type. That's because this portfolio size, despite it actually being a number, is still formatted as a string. So we need to wrap it in a float function.

This gives us twenty thousand dollars. Awesome. Now we need to loop through the every row in the pandas data frame and modify the value that's contained in the number of shares to buy columns.

So to do that, we're going to say for row in final data frame index. And then we are going to say final data frame dot LSE. This is the exact same method that we used to modify this before.

We're going to do row and then number of shares to buy. Then we're going to use the assignment operator to say divide our position size by that stock's stock price. So if we put pricing here, that should be the trick.

Now we'll print out our final data frame to make sure that this is operating correctly. This looks good. Let's just compare two stocks here.

(3:25:12 - 3:25:35)

So we have a stock at thirty five dollars and a stock at eighteen dollars. So this is interesting because this is roughly double this. So this should be roughly double this and it kind of passes the eyeball test.

So that's good. Now the only other thing we need to add here is the floor function. So as you'll recall from the last section of or the last project of this course, you can't usually buy fractional shares at some brokers.

(3:25:36 - 3:26:23)

So if we wrap this in the floor function, it will give us the closest lower integer to the number of shares that it calculated. So if we run this, we can see that those have all been transformed to integers. Awesome.

We are now ready to move on to building a better and more realistic value strategy. All right. So it is now time for us to build a better and more realistic value strategy.

Now, just like in the momentum project that we built earlier in this course, what we're going to do to make a better value strategy is to build a strategy that takes into account many different metrics. And the reason why that makes the strategy better is because every valuation metrics tend to have certain flaws. For example, as we saw earlier in this project, the price to earnings ratio doesn't exactly work that well with stocks that have negative earnings.

(3:26:24 - 3:27:07)

Similarly, another popular valuation metric is the price to book value ratio. And that ratio doesn't work well with stocks that perform a bunch of share buybacks. So there's lots of different things to consider for every valuation metric.

And one common way that investment managers reduce any of the flaws, reduce the impact of any specific valuation metrics flaws is by creating value strategies that take into account a composite of value. So instead of just saying we're going to rank stocks on price to earnings ratio, they rank percentiles for a bunch of different valuation metrics and then rank stocks according to the average of those percentile scores. So that's exactly what we're going to do in this section.

Here are all the ratios we're considered. So price to earnings ratio. We already saw that.

(3:27:07 - 3:29:48)

Price to book ratio. Now, this is the company's stock price. The company's stock price divided by its shareholders equity per share.

The price to sales ratio. This is the company's stock price divided by its revenue per share. Enterprise value divided by EBITDA.

Now that is a more complicated valuation metric. EBITDA stands for earnings before interest taxes, depreciation and amortization. Enterprise value is basically the number that you would have to pay to buy out the company in full.

So that doesn't just mean buying it all of its stock. You would have to also buy it all of its debt and any minority share holdings that exist. So that's what enterprise value is.

And then another common valuation metric is enterprise value divided by gross profit. So all of these five valuation metrics we're going to now pull in from the IEX cloud API. Now to do that, we're going to start by just making a single API call and then we will loop through that same API call for a bunch of different stocks that are listed in our stocks data structure to build a pandas data frame that contains all the data.

So to start, we'll just do this for one symbol and that symbol is going to be Apple. And then we're going to do a batch API call here. And unlike before, we're not going to use a batch API call because we want to query the IEX cloud API for multiple symbols.

But instead, we have to use a batch API call because we're querying the API for multiple endpoints. So since the different valuation measures listed here are all so different, they're not all actually going to be stored in the same endpoint. So we need to find where they are in the IEX cloud API and then add those endpoints to a batch API call URL.

So let's start by just creating the variable batch API call URL. And just like before, this is going to be an F string. Now let's to make this easy, we're just going to base this off of the batch API calls that we executed earlier in this project.

So let's scroll up and find that. All right. So here is where it is.

I am going to highlight all this and copy it to my clipboard. I'm going to scroll back down to where we were just working and paste this in. Okay.

Now we have to make a few changes here. The first is that we need to replace this symbol string with just symbol and then let's actually execute an API call against this to make sure that it's working properly. So we've seen this many times by now, but we're going to do data equals and then to define the variable we're going to say requests.get and then we're going to pass in that batch API call URL not the bad URL.

(3:29:50 - 3:31:18)

Batch API call URL. And then let's do print data.status underscore code and if this returns 200, then we know it's working correctly. That is awesome.

Okay. So let's go through all of these metrics one by one. I'm actually going to copy these and paste them down here and comment them all out.

Oops. There we go. And then underneath each of these comments we're actually going to parse

that value from the IS cloud API.

So the first thing we need to do is get the price earnings ratio and we actually saw how to do this in the last section of this tutorial. So if you go back up here, we are going to go to where we copied this original batch API call URL. And we are just going to grab this parse here.

So it's data and then symbol and then quote and then PD ratio. So let's copy that and scroll back down to where we were just working, which is right here. And if we print this, this should give us what we need.

Okay. Now we're getting this error response object is not subscriptable and that's because I haven't called the JSON method on our API call. So we're going to add that here and then we're going to remove this data.status code because that's now going to return an error.

(3:31:18 - 3:32:12)

Let's run this code cell again. Great. So this tells us that Apple has a current price earnings ratio of 38.84 according to the IEX cloud API.

Let's now assign this variable to this value to a variable called PE ratio and we're going to do similar things down here. So PB ratio is going to be price to book ratio. And for now we're just going to assign that a value of not a number.

So this is a not a number data structure that's stored within the NumPy library. If you've never seen that before. And we're going to PE, or sorry, PS ratio.

Same thing, NT.NAN. And then down here we're going to do EV to EBITDA. Same thing, NP.NAN. And then we're going to do EV to gross profit. EV to gross profit.

(3:32:13 - 3:33:33)

Equals NP.NAN. Okay, we've now filled these all with placeholder values of NP.NAN. Let's now go into the IEX cloud docs and figure out where these data points are contained within the IEX cloud API. So just like before, all I did for this was I Google IEX cloud docs and it's the first non add hit that I'm clicking. Alright, so now I'm going to do a control F search for PB.

And with any luck, this will show us where the price to book ratio is held. There were seven hits. I just searched through all of them.

It wasn't any of them. So now I'm going to try price to book. It only gives two hits and both of those hits are this value score, which looks like it's its own composite metric here.

So let's try price to book with hyphens. There we go. Price to book.

What endpoint is this? So this is the advanced stats endpoint. And to add that to our batch API call URL, I'm just going to copy it. And I'm going to add it here right after the quote.

So comma advanced stats. So now we are going to print our data object and see what we have to parse out. So do that.

(3:33:34 - 3:33:56)

Alright, so we're going to have to do multiple levels of parsing here. We're going to print data, Apple, and then within that we are going to parse out the advanced stats endpoint. Alright, so now when we run this code sale, this will show us what's contained inside the advanced stats endpoint.

(3:33:56 - 3:35:01)

We want PB ratio. So I'm going to do a control F for PB here and see if it highlights anything. I don't see it there.

So I'm going to do a control F for book and I see it right there. So price to book. Now we can copy this in and parse it out.

Alright, so this tells us that Apple has a current price to book ratio of twenty three point eight eight. I'm going to copy this and assign it to our variable of PB underscore ratio. Then what I'm going to do is I'm going to remove the final section of parsing from this line.

I'm going to print out the JSON object again, and I'm going to try to find the price to sales ratio. So just like before, I will start by saying PS. There is one hit within this data structure, but it's TTM EPS.

If you've never seen that before, all that stands for is trailing twelve months earnings per share. And that's not what we want. So I'm going to do a control F search again for sales and you can see right here we have price to sales ratio.

(3:35:01 - 3:35:45)

Awesome. So what I'm going to do is I'm going to copy this price to book ratio data point. And instead of price to book, I'm going to change this to price to sales.

And then I'm going to print the PS ratio so that we can see what value we get. So this tells us at the bottom here that Apple has a current price to sales ratio of eight dollars and point or sorry eight point two eight. So I can delete that.

Now we have to parse out enterprise value to EBITDA and enterprise value to gross profit. Now both of these are a little more complex but where you could still make it work. The reason they're more complex is because the V to EBITDA ratio and the EBITDA gross profit ratio they aren't actually provided explicitly by the IEX cloud API.

(3:35:46 - 3:35:56)

Instead we're going to have to pull in their constituent metrics. So enterprise value and EBITDA or enterprise value and gross profit respectively. And then we're going to have to do the division ourselves.

(3:35:57 - 3:36:24)

So to start let's try and find enterprise value. We'll create a variable called enterprise value. And to start we will assign that a value of NP dot man.

And then we're going to want to do a control S search within this endpoint for enterprise. And interestingly enough the IEX cloud API does provide one of the enterprise multiples but it's enterprise value to revenue which is not one of the ones we're going to use. So instead we're going to pull in this enterprise value endpoint.

(3:36:25 - 3:36:38)

We're going to grab this for price to sales and we're going to copy it down to the enterprise value variable and then replace this with enterprise value. And then let's print it. Now since Apple is a very large company this enterprise value should be huge.

(3:36:39 - 3:36:51)

Yeah it looks like it's more than two trillion if I'm just eyeballing it there. Actually let's divide that by a trillion to see if I'm right. One and then we should be nine zeros one two three four five six seven eight nine.

(3:36:52 - 3:37:13)

Sorry not nine zeros twelve zeros so we need three more one two three. And it looks like it's around two point two trillion dollars of enterprise value for Apple right now which is pretty crazy. All right so we have our enterprise value.

Now we need our EBITDA. Let's do a control F search for EBITDA and see if it pops up in this endpoint. You can see right there that it is pretty easy to access.

(3:37:14 - 3:39:05)

What we'll do is we will just copy this paste it there. Change the variable name to EBITDA and then change the parsing to EBITDA as well. It's all capital so make sure you capitalize this as you're working through it.

And then for our EV to EBITDA ratio what we're just going to do is say enterprise underscore value divided by EBITDA. And that ran correctly. What we need to do now is print it to see what value we're actually storing EV to EBITDA.

So this tells us that Apple has a current EV to EBITDA ratio of twenty eight point two. Awesome.

So using this information we can now proceed to calculating our EV to gross profit.

We already have our enterprise value and it's stored in a variable up here. So what we need to do now is create a variable called gross profit and parse out the gross profit number from this from this JSON object. So to do that I'm going to do a control F search for gross and you can see the gross profit metric is right there.

So what I'm going to do is copy the parsing that we did for our EBITDA variable earlier in this code cell. I'm going to paste it here and then I'm going to replace the EBITDA parsing with a gross profit parsing. Now if we print out gross profit what does it give us? This shows you that Apple had gross profit of I'm going to say that's ten billion.

I'm not really sure just by looking at it there's no commas which makes it a little harder to read. Anyways the number is being pulled in successfully and that's the main thing. So now what we're going to do is say EV to gross profit is equal to enterprise value divided by gross profit.

And then we will print this on the next one. I'm going to print EV to gross profit and I'm actually going to run this code cell again without that statement. So this shows us that Apple has an EV to gross profit ratio of twenty two point six.

(3:39:06 - 3:39:45)

Okay so we've now done all of the parsing and calculations that we need to get all of our value metrics for one stock. Now it's time for us to generalize this logic and build a pandas data frame that stores all of this information. So we'll do that next.

Now you'll see in the header for this code cell that it says you'll notice that I use the abbreviation RV often and that's sort of the acronym that we're going to use for robust value which is kind of the name I'm going to use for this investment strategy. It's kind of the equivalent to high quality momentum that we used in the last project. So as before the first thing we need to do is say RV underscore columns and then we need to create a Python list that defines all of the columns that we're going to use in our pandas data frame.

(3:39:45 - 3:40:35)

So the first thing will be ticker. That's the stock ticker of the company we're looking at. The next thing will be price and that is the stock price of the company we're looking at.

The next thing is number of shares to buy. Just like in our first two projects that tells our trader team how many shares of the specific security to buy when they're rebalancing. And now we're going to start working through all of our valuation metrics.

So more specifically we're going to say price to earnings ratio. And then we're going to say a percentile score for price to earnings ratio. So we'll call that PE percentile and then we'll, we need a column for our price to book ratio, so price to book ratio and then we need a column for

the percentile score of price to book, so PB percentile.

(3:40:38 - 4:33:02)

And then we need a column for our price to sales ratio. So that's priced to sales ratio. and then we need price to actually we'll say PS percentiles and that matches the abbreviation that we use for our first two percentile scores and then we need enterprise value divided by earnings before interest taxes depreciation amortization so so for that we're just abbreviated right from the get-go because it's quite long and we will say EV to EBITDA and then on the next line we will do a percentile score for that metric so EV to EBITDA percentile and then we need our EV to gross profit so we'll say EV to GP and then EV to gross profit percentile the last thing we need to do is calculate our robust value score so we'll do that right from the get-go this time our RB score awesome I'm gonna run this code so just to make sure that the syntax here is all working that is awesome we can now move on to creating an empty pandas data frame so we'll call this data frame RB data frame and that stands for robust value data frame and to associate it we're gonna say PD data frame and then inside of this class call we're gonna say columns equals RV underscore columns and what does that look like it doesn't look like anything because I didn't get it yet so let's add our V data frame on the last line and as you can see this has all of the columns that we specified and we are ready to start creating our loop now we have already created our symbol strings object earlier in this project so if you scroll up here a little further what am I looking for here it is so this code cell was already provided when you guys open this up and this has created symbol strings that we can loop through when creating our data frame so we don't have to do that again because those variable values are already stored in this Jupyter Notebook so we're ready to create the loop what we need to do now is say for symbol string in symbol strings and to make sure this is looping correctly let's just print out the symbol string awesome that looks good and then within this loop we have to create our batch API call and we're gonna do that in the same way that we did before we're first going to create the URL and then once the URL has been created we're going to use our requests library to actually execute the HTTP request so we'll call the URL batch API call URL and then we already created this URL a few cells earlier so we can just go up to here and copy this with a few minor changes so the minor changes I was referring to are that we have to replace this symbol with symbol string and then it should be good to go to make sure this is working properly I'm just gonna print the batch API call URL and let's see what happens awesome you can test easily whether any of these is working by just clicking on and what that will do it was in a separate browser window it will open up the JSON object that you'll be pulling in so let's see awesome in a browser window like this this is pretty useless because you can't really manipulate it but that's a quick easy way to tell that your API calls working right now let's execute our HTTP request so we're gonna do that in the exact same way that we did before data is equal to requests.get and then inside that get method pass in batch API call URL and then we're gonna print out data.status code and since we already clicked on the URL to open up the JSON object in a separate tab a few moments ago we know that these API calls are working but printing the status code is another way you can check it so if these API calls are functioning as we want

them to this should print out HTTP code of 200 for each API call so let's see one two three four five six and there should be six so that is awesome all right so now we're ready to start parsing our data frame and to do that the first thing you need to do is add the JSON method to the request.get method and what this does is it turns this object which is a request response object into a JSON for us to manipulate now if you print out the data for each one we'll get a bunch of big JSON objects and this is what we need to start manipulating so now that we're in here we need to loop through every ticker within our symbol string and append the data for that ticker to our empty pandas data frame to do that we need an inner loop and that inner loop will look like this we'll say for symbol in symbol underscore string split and then we're going to split on the common character just like we did in the other two projects to make sure this is working properly let's print out our symbol awesome so you can see we're going through all of the symbols in our stocks pandas data frame that we created at the very start of the alphabetical order okay so now that that's done we can start using the append method to add data to our empty pandas data frame and to do that what we're gonna say is our V data frame is equal to our V underscore data frame dot append now we're gonna use same we're gonna use similar logic inside of this append method as we've used earlier in this course so we need a series and then inside of that pandas series we need a Python list and we also need to specify index is equal to our V columns now in case you're not remembering what this does this index equals our V columns specifies which columns of the empty pandas data frame each item of this pandas series should be appended to and then inside of the pandas series we want to add the ignore index equals true now I haven't really done a super guitar explain this but to be very specific what this does is it means that every row that you add to the pandas data frame will have its index column automatically calculated for you so the first row will have index 0 the second row will have index 1 the third row will have index 2 the 50th row will have index 49 and so on so that's what this ignore index does now within this pandas series we need to parse out all of these data points so like I did in the last project what I'm going to do here is I'm actually just going to copy these in and since it's a comma separated list we can just replace each element of this list with the data that we want to parse out so we'll do that one by one ticker we want to replace with a symbol and remember that symbol is the incrementer of the inner for loop that we're building here for price we want to parse it with price and we saw how to do that up here that's actually not true we didn't see how to do that there but what we can do is go back up to our earlier project where we did this for for just a screener that only factors in price earnings ratio and we can copy this parsing that we used here so we'll do that and scroll back down here and then we want to replace this price with that awesome now number of shares to buy just like in the in the other projects that we work on in this course we're just going to replace with a placeholder value and a price earnings ratio we actually parsed in the last code cell along with all of the other valuation metrics that we need so what we can do here is just say copy that and add it where the price earnings ratio is and then we come to the section where we need price to earnings ratio percentile and all of the percentile scores in this pandas data frame just like in the ones that we worked with earlier in this course we're going to replace them with a placeholder value of NA because in order to calculate percentiles you need the pandas data frame to be fully populated first so we will calculate these a little later on in this

project for now we're going to replace them all with NA and one more and for that matter this robust value score in the last column also needs to replace with NA so let's do that now too all right the next thing we need is our price to book ratio and just like the price to earnings ratio we had parsed that out in the last code cell so I'm going to copy that I'm going to add that here oh sorry not there down here I knew something didn't look great so I got there we're gonna do the same thing with our price to sales ratio so that's right here all right now for EV to EBITDA and EV to gross profit we actually have to do some special logic here because there are metrics that are used in both of those calculations that aren't available for every stock so here's how we handle that the first thing we need to do is pull in our enterprise value our EBITDA and our gross profit metrics and we're going to do that it was side of this append method so there's enterprise value here's EBITDA and here is gross profit awesome and then in here for now I'm not going to do this special logic that I just referred to earlier because I want to show you what happens if you go into it oh and this isn't EBITDA it should be enter price value and then here we're going to enter price on this value divided by gross profit all right let's run this and see what happens so far so good 72 and then we get a key error for a P L and that's because all of those Apple instances need to be replaced with a symbol let's do that try that great we have another key error because there's more instances of Apple up here that is embarrassing like I said earlier in this course I'm not going to edit this out because I want you to see how I debug stuff in real time it might be useful to see the debugging process of a different developer okay what happens now here's what we're getting so it says unsupported operand type for division integer and none type and that's referring to this line right here okay what's going on let's print out our enterprise value so we have a value for enterprise value now let's print out our EBITDA none now that is the source of the error so to fix this we need to find some sort of workaround to fill this value even when there's a none type for one of the metrics that it contains now there's lots of different ways that you can handle this but what we're going to do is this I'll show you so we saw when building our portfolio input function in earlier projects have this course that try accept statements can be used to handle exceptions so that's what we're going to use we're going to use try this and if that doesn't work we're going to make an accept statement that handles specifically the type of error one counter which is a type error and then we're going to say MP not and now because this is in on a pen statement we actually can't put it there instead we're going to create an EV to EBITDA ratio and put this outside of the append statement I'll show you more about what that looks like in second after I create these okay so this is going to be put inside the append statement right here after we create all these and since the indenting is in a loop I have to tab that in and then we will replace this with EBITDA now okay what this does is it tries to calculate EVD EBITDA using this and if that fails because of a type error then it will fill it in with NP dot NAN now that's not good because NAN stands for not a number and how are you supposed to source dogs on value metrics that aren't numbers well we're going to see later on in this section or later on in this project I should say that there's a number of different ways that you can handle missing data in the pandas library to fill in that values that aren't there with reasonable values by performing calculations on the other values in the data set so I will see more about that later but the main idea to remember now is that even though we're filling our pandas data frame with null values now those aren't going to stay

there for our final calculations because we're going to replace them with something more meaningful later okay the next thing we need to do is we haven't seen it yet but this type error happens with the gross profit metric as well because some companies don't actually report gross profit depending on how what what their operations look like in those sorts of things so what we're going to do is create a similar EB to gross profit metric here and use the exact same type error exception handling to to handle this okay so if you two gross properties will enterprise value divided by gross profit this looks good the only other thing we need to do is take this variable and replace the actual divisor in the expense data okay now I'm going to delete this other code cell that we were working with and run this again to see if it works so far so good awesome so that code cell ran with no problem let's print our data frame now see what it looks like awesome so we have ticker stock price number of shares to buy price to earnings ratio PE percentile price to book ratio PB percentile price to sales ratio PS percentile Evita percentile Evita GP percentile and the RE score everything looks like it was added properly here so we can now move on to dealing with missing data in our data frame now our data frame contains missing values because we use that try accept logic in the last section to fill in missing values whenever the IES cloud API provided a non response for a metric that we were using and that's basically what this is it says our data frame contains some missing data because all the metrics we require are not available through the account we're using so to see where this missing data is what you can do is use the pandas is null method so this is kind of the the statement you execute to see all call sorry all roads that have missing data in our data you say RV data frame and then in square brackets you would say RV data frame is null and then on that isn't all you want to say any and then access equals one okay so to break this down we're taking our RV data frame and then we are filtering in the square brackets to say any part of RV data frame where the is null method is true and then on that we want to apply to any columns and this access equals one tells us columns not rows so when we run this this is going to return a modified version of our pandas data frame that only contains rows where there's missing data alright so as you can see there is quite a bit of missing data and you can actually see how much there is by calling the length of this so there's 55 rows that have some sort of missing data in the pandas data frame which is about ten percent of the entire data set so let's look at this again as you can see all not all of the indices of the pandas data frame exists so it starts at 17 18 20 26 there's lots of missing rows because all the missing rows have no missing data so let's take a look at this first row which covers the stock Aflac which is an insurance company there is a non-value here for price to sales ratio there is a non-value actually that's the only non value for that column now if you look in other columns like this column there's a non value for price to earnings ratio and overall these non values aren't super problematic because in the next section we're going to see how to deal with it so this says dealing with that dealing with missing data is an important topic in data science and there's two main approaches you can either drop the missing data using the drop NA method or you can replace the missing data with a fill in a method so the fill in a method is pretty interesting because you can actually pass the information that allows it to fill in missing data with summary data from that same column so as one example if we have missing data in our price to earnings ratio column we can fill it in with the average value of all price earnings ratios for all the other stocks in the data sets so

that's exactly what we're going to do we're going to replace missing data with the average non null data point from that column so here's the code to do this I need to create a for loop that loops over all of the columns in our pandas data frame so we'll say for column in RV underscore data frame columns and then let's just print this out to see what's happening great as you can see this is successfully looping over our pandas data frames and then what we want to say is RV underscore data frame dot fill and a now if you've never used this before one easy way to see which parameters it accepts is to mouse into the the parentheses of the method and then hit shift tab and this will pop up a little bit of documentation so what we're going to do is we're going to say fill in the missing values with the average value from that column so to get the average value we're going to say fill and a and then we're going to say RV underscore data frame we're going to select that specific column from the data frame and then we're going to call the mean method on dot mean and one other thing I forgot to add here is that we don't want to apply this to the entire data frame we want to apply to this specific column of that data frame now the reason why you have to add this is because if you didn't for the first iteration of this loop when it looks to the ticker column it would fill the entire data frame with the average value of the ticker column which is not what we want so if we run this we get a value error and says could not convert string to float now the reason why this is is because you can't actually so what this is doing is it's trying to calculate a mean for the ticker column which is obviously impossible so we actually need to create a subset of our columns and to do that the easiest way is probably to just print out our columns to RV data frame columns we will grab this and then we need to modify our for loop so that it only loops over certain columns of the pandas data frame so we will put that there and then does it still work with it being multi-line it does awesome so we want to take out all of our non numerical columns so that would be all the percentiles it would be PB percentile PS percentile EV to even data percentile EV to GP percentile and RV percentile and I'm actually going to take out this price column too because there's not going to be any missing price data all right so if we run this this shouldn't look a these all in one line just for readability there we go all right what happens when I run this awesome now if we run our same command from earlier to identify missing data in our data frame if we have done this properly this should return an empty data frame because there should be no values anymore and there still is why is that this price to sales ratio still has quite a bit of no values why is that oh I see so within this fill in a method you actually have to pass the in place true and this says don't just return the modified column modify the original column instead so we'll do that and then when we run this we get an empty pandas data frame which shows us that we have successfully dealt with all the missing data in the original data frame so that's kind of a high-level introduction of how to deal with missing data in pandas there's lots of more information to learn about this and deal with missing data is a full topic in and of itself so this is just a high-level introduction we can now move on to calculating value percentiles all right so what we need to do now is we need to calculate value score percentiles for every stock in our universe more specifically we need to calculate percentile scores for the following metrics price to earnings ratio price to book ratio price to sales ratio Evita Evita and Evita gross profit now unlike before we can't use the start of every string as a way to interpret in that string because in the last project we had the same two words at the start of every momentum metric

like an example would be we had one year price momentum and one year return percentile so we can't do that anymore because we use some abbreviation so if you look here you can't use price to earnings ratio to find the price earnings ratio percentile column because there's an abbreviation so I did that intentionally so that you could see a different way of approaching this problem what we're gonna do instead is I just copied the column names to my clipboard like this and we are going to turn this into a dictionary we're gonna call this dictionary metrics and here's what we're gonna do so we need to wrap this entire thing in curly brackets and for every metric column that column name is going to be a key for the dictionary and the percentile column is going to be the value of the dictionary like that so if we do that same operation on a few different lines this will fill it pretty quickly all right let's put these on their own lines like that okay so basically what this does I'm gonna run this code so to make sure it's formatted properly and the way we're gonna use this dictionaries we're going to pass in a a metric and it will return back the name of the corresponding percentile row so to calculate our percentile scores we are going to loop through every metric in this metrics dictionary so here's how we'll do that we'll say for metric in metrics dot keys and then within here we need to calculate percentile scores and we need to do that for every row so we need to create an interlet that says for row in rv underscore data frame dot index and then here's where we're just going to print the row now this should print how many metrics other five five times this should print zero to five hundred and four so we have one two three four five awesome okay now here we need to use the pandas loc method to access the percentile score columns now here's how we'll do that we'll say rv underscore data frame dot loc and then the first entry of this loc method will be the row and the second entry will be the column name and that will be metric no sorry that won't be metric that will be metrics hat metric and this funny looking dictionary access statement is basically pulling in the percentile column to make sure that that's working properly let's just print this and what this should do is just print a whole bunch of NAs awesome so this shows that we're accessing all of the different percentile columns and what we want to do here is use the percentile of score to calculate a percentile so to do that we'll say that is equal to stats dot percentile of score and then within this we're gonna have two items so this will be the entire column and this will be the value of that row at the column so to get the entire column we're just going to say rv underscore data frame and then we're gonna pass in the name of the column so that will be metric and then over here we need and then over here we want to get the value of the row at that column to do that we're gonna say rv underscore data frame at loc and instead of saying the metrics dictionary at that metric we're just gonna say the metric so let's run this and then print our pen this data frame to see what that looks like great so here we have all the percentile scores now you can see this appears to be right because here we have a low EV to GP and it has a low percentile here is a high EV to GP with a higher percentile and you can see the same logic for all of these now just like before this Python line is very long it doesn't actually all fit on one line so I'll show you how you can easily make that a little cleaner we'll import the function specifically so we'll say from scifi dot stats import percentile of score as score and then we can replace this with just score and actually since we use a different dictionary method this now fits on one line and is much more readable if you run it they'll give the same output great so that's how we calculated our percentile scores as a quick recap of

what is done in this code cell we imported the percentile of score function from the scifi dot stats module and we named it score and then we created a dictionary of metrics where every key in the dictionary is one of the valuation metrics that we're using and every value of the dictionary is the corresponding percentile column of our pandas data frame and then we created a double nested for loop where the first for loop loops over every metric in our metric dictionary and the second for loop loops over every row in our pandas data frame and then within both of those for loops we have an operation here that calculates the percentile score of every valuation metric and assigns that value to the corresponding percentile score column and at the very end we just printed our pandas data frame now we can move on to calculating our least score so just like before we're going to use the mean function from the built-in Python stats module so we'll say from stats import mean and then we're just going to calculate the mean of five and ten to make sure that this is working properly we say no module named stats that's because I should have said statistics great so the mean of five and ten is indeed 7.5 so that seems to be working properly now what we need to do is we need to loop through every row of our pandas data frame and calculate the the average calculate the RV score for that row so to start we'll just say for row in RV underscore data frame index and let's just print that row so this should print a bunch of numbers from 0 to 504 so it goes 0 all the way to 504 that is excellent now we need to access this row of the pandas data frame and like before we can do that with the LLC method so what I'll do is I'll just copy this paste it here and then I'll change which column we're accessing to RV score and to make sure this is working properly let's just print this all right that looks good now what we need to do is access all of the different percentile scores from our pandas data frame and calculate their mean so on this side we will say mean and then since this will be a fairly long line I'm actually going to create variables for all these so PE ratio, PEV ratio, PS ratio, EV to even dollar ratio, and EV to gross profit now I need to grab the LLC method from up here actually I just thought of a better way to do this what we're going to do is we're going to delete this we're going to create an empty pandas list and we're going to call that value percentiles then we want to grab this same for loop here so for metric and metrics.keys and we are going to say grab the value from the corresponding percentile awesome we want metrics all right we are going to now just do this over one iteration of the loop to make sure it works properly and before we do that we actually have in here to calculate the mean of this value percentiles and outside the loop we are going to print the RV data frame awesome now one other thing we can do to test this actually is print that value percentiles all right so to make sure this is working properly all we have to do is compare this to this you can see they're the same we have to compare this to this you can see that the same we have to compare this to this and you can see that they're the same and we have to compare this to this and this to this awesome so that is working properly and what we can do now is delete these print statements make this loop iterate over the entire pandas data frame and then inside the loop we can print the RV underscore data frame awesome you can see all of our percentile scores were correctly calculated and now we can move on to calculating and now we can move on to selecting the 50 best value stocks great so as before we can use the pandas sort value method to identify the 50 best stocks in our universe according to the RV score column so to do that we access our RV data frame we call the sort values method on it and then

in here we have to pass in a few things you have to pass in the column we want to sort on and then we have to specify the ascending method and we want this ascending to be true because we want the lowest valuation metrics at the top and then we have to specify in place once that's done we can print our pandas data frame outside of the sort values getting like this to see what modifications have been made alright so now you can see the cheapest stocks according to a basket of different valuation metrics we have a Kohl's we have FTI I'm not actually sure what company that is but we can find it pretty easily FTI and it's Technic FMP it looks like they're a global oil and gas company it makes sense that they're cheap right now because oil prices are so low HFC is Holly Frontier Corporation they are also an oil and gas company an AL is an airline they are priced very cheaply right now because the entire travel industry has experienced significant headwinds due to the COVID-19 pandemic and HP I believe is Hewlett-Packard I know the actual Picard enterprises and they make printers and those sorts of things they've kind of always traded out a pretty cheap valuation so that's that on the other end of the spectrum you can see what stocks are trading at extremely high valuation so DX CM I believe is Dexcom they make continuous glucose monitoring yeah they make continuous glucose monitoring systems for diabetes management now I'm not sure what stock that is but we can find out this is ServiceNow and they're an American software company every software company in the world is trading at pretty high multiples right now then we have AMD they make CPUs and other sorts of computer hardware just like Nvidia and then PAYC I think is Paychex which is a if it is Paychex they are a payment processing company no Paycom although they're in the same business it's an American online payroll and human resource technology provider based in Oklahoma City okay so that's done what we need to do now is we need to drop all of the stocks that are after the 50th index so to do that we will say RV underscore data frame is equal to RV underscore data frame at index 0 to index 50 and I actually didn't print that so what we can do now great so this looks like it's been truncated to only contain the 50 cheapest stocks in our universe to make sure that there's like 50 what we can say is we want to calculate the length of the index of the different and that is 50 awesome the only other thing we need to do is we need to filter or start not filter we need to reset the index of the payment to stay the thing we saw before that we do reset underscore index and then we want to pass drop equals true into it to avoid duplicating the existing index of the pandas data frame and then we also want to pass in in place equals true and like all the other instances including right here what this in place equals true parameter does is it means we're gonna actually modify the original pandas data frame instead of just returning a modified version temporarily so we're gonna score data frame and see what this looks like awesome so now our index goes from zero and should finish at 49 yep and these are the 50 cheapest stocks in our universe so now that we've selected the 50 best value stocks we can move on to calculating the number of shares to buy now earlier in this Jupiter notebook we defined a portfolio input function that will allow us to easily create our portfolio size so to do that we will say portfolio underscore size we will run this for sale and then we will enter a string object is not calling well why is that that's because we need to pass in portfolio input in our portfolio size all right now we want to enter the value of our portfolio so that's going to be a million dollars so one and then six zeros one two three one two three actually let's spice things up a bit and do something different so we'll do two and a half million

so two five and then five zeros one two three four five awesome now let's print our portfolio size object or variable I should say to make sure that it's stored correctly awesome now we need to calculate our position size and we've done this twice now first in our equal weight S&P 500 strategy and then in our momentum strategy so you should be good to do this on your own I would encourage you to pause this video and try to do it by yourself but if you want to follow along you can so to do this we will say position size is equal to and then the numerator of this division is going to be the portfolio size variable that we just created and the denominator is going to be the length of our pandas data frame so I've mentioned this in earlier videos but I prefer to just do length of the RV underscore data frame index and then we can print our position size now what does this give us an unsupported operand type for string in it that's because I actually ran into this problem in the momentum project as well we need to force this to be a float now what is this giving us $50,000 and if you think about it our portfolio is two and a half million dollars and there's 505 stocks in our sorry 50 stocks in our universe so if we want to make sure that this is right we can just test using the double equality operator whether the position size is equal to two and half million divided by 50 and if this returns true then we know that the calculation has been done properly awesome now we need to figure out how many shares of each stock we need to buy to reach a position size of $50,000 in each stock so to do that just like before we're going to create a for loop that loops through every row of our pandas data frame so we'll say for row in RV underscore data frame index and to make sure that's living properly we'll just print them it should go from 049 and it does which is awesome and then we will say RV underscore data frame dot LLC and in here the first thing that we need to put into the row the second thing we need to put in is the number of shares to buy column and to make sure that's working properly let's just the reason I always test these LLC methods is because sometimes if you type the column title wrong it won't work and it looks like that's exactly what happened here so let's see what happened we have number of shares to buy and to see what the difference is I'm going to scroll up to where we originally defined that column title and I'm going to paste it beside it so that's way up here and so we have number of shares to buy number of shares to buy huh they look the same to me maybe that's not actually what the error was let's go back down and actually read the error message that could help hey during handling of the above key error okay let's print out our column names to see what's going on number of oh my soul it is it is just a typo I didn't put an S on shared so that's the problem and now this prints out all the entries in our number of shares to buy column of course that's not what we want we need to actually change the value of those cells in the pandas data frame so to do that we'll use the assignment operator the equal sign and then we need to do a division so what goes on the top it's our position size and then in the numerator we need to figure out our stock price so to do that we will use another LLC and paste it there we will replace the column with price and that should be all we need to calculate the number of shares we need to buy so now the best time let's print our RV data frame and run this code cell great so we can see here that for Kohl's it says by 2,000 shares for FTI it says by 6,500 shares and you'll notice that since we used a bigger portfolio size these numbers are just broadly larger than when we used a 1 million dollar portfolio size which makes sense you can also quickly sanity check this by looking at this stock it has a lower stock price than this stock and

because of that you need to buy many more shares of that stock to reach the required position size so everything looks to be good here the only other thing we need to modify is to say math floor and what this does is it rounds every entry in that number of shares to buy column down to the nearest integer for all the reasons that we described in the first two projects of this course so that is done we can now move on to formatting our Excel output great so just like before the first thing we need to do is create our writer object and to do that we will say writer is equal to PD Excel writer and then in here we have to pass in a few things the first thing we have to pass in is the name of the excel file we want to create which is value underscore strategy dot xlsx xlsx not xslx and then the next thing we need to pass in is the engine the engineer we use here just like before is xlsx writer now to make sure this is working properly you can just do the writer dot save method that we have learned about earlier and then if you go into the folder that we're working in starter files you should see the file here and it should just be empty so here it is and if you click on it it's just an empty Excel document which is awesome because we haven't done any work on it yet so what we need to do now is we can delete this save method from that code cell then we need to access our RV data thing and we use the to excel method to pass in the name of to pass in the pandas data frame to the excel file that we're working with now this takes three things you have to say writer and then you have to say she underscore name is equal to value strategy and then you have to pass in index equals and this just means that the index here won't get passed into the pandas data frame so we'll do all that that creates our kind of save frame the next section is that we need to create the formats that will need for our xlsx file and you'll recall from our first project that we've already have lots of practice with creating these formats and since we've already done this we're just going to run this code cell and then continue through this tutorial so the next thing we need to do is actually apply these formats to each of our columns and to do that we're going to start by creating a dictionary and then we will loop through that dictionary afterwards so let's go up here to where we first created the columns of our pandas data frame and from there we will paste them in and create a dictionary so here's all of our columns we will scroll to the bottom paste those in here we'll get the indexing set up properly and then we will say column formats equals and there is that I'm going to add this to its own line and then I'm going to put this ticker on a new line and then I'm going to get the indenting all set up nicely awesome that looks a bit better what we need to do now is actually format this like a dictionary so I'm going to use the multi cursor functionality of this jupyter notebook to add my cursor to every one of these and then I'm going to add some boilerplate each of these lines so the first thing we're going to do is specify a column I'm going to put a on each line and then we will fix it afterwards then we need to put the semicolon which define basically is the defining characteristic of a Python dictionary and then I'm going to create a Python list and to do that we're going to move every person to the end of each line there we go move back one and then add the closing list character the other thing we're going to do is move back another one and add in a comma and then string underscore format now this worked well for every every line except for a few so it's always good to go through and just sanity check these that one needed a bit of work and this one here something wrong with it that used to go there and we can delete that okay I'm going to just run this post out to see if this is formatted properly it's not string format is not defined

okay that's because it's called string template that is not good what I'm going to do is do a find and replace so I haven't showed you guys this yet but it's an interesting feature of the jupyter notebook if you hit edit here and then scroll down to find and replace what I'm going to do is I'm going to search for a string underscore format and replace all of those with string underscore template boom that fixes that problem now if I run this code cell code cell we should be good to go and we are so that is awesome what I need to do now is change the column letters for each of these so I hope you guys all know your alphabet will go a b c d e f g h i j k l n alright now we can actually move on to formatting our our excel file so what we'll do to start is we will say for column so what this does is it loops through all of the column letters that are keys of this dictionary and to make sure that's working properly let's just print out all the column letters it loops from a through n and the dictionary goes from a through n so that is good now what we need to do is we need to call writer dot sheets and then we need to access the sheet that we named our pandas data frame and that is value strategy so we will copy that we will move back down here and we will fix that in there and then we would call the set column method and you'll recall this takes a few different parameters it takes the column so that will look like that it takes the column width so we'll use 25 pixels and then it takes the format so that will look something like this string underscore template now I'm gonna run this loop and you'll notice that the statement that is being run with each loop doesn't actually change with response to the loop so that this is just going to make sure that this is working properly for column a and then once that's done we can go in about changing the values for each iteration loop I'm also going to call the writer save method to save it and we can now look in our working directory to see if this has formatted column a successfully and it has so we can now move on to interpolating some values into this so here we want to use column and we need to change this to an F strings that actually interpolates properly we need to use column there and then here we need to access the second item of this list so to do that we will say I can't quite scroll this up enough to get it away from my webcam so here we want to say column underscore formats at column so this gives us the list and then we want the second item of that list so we'll try that and now we'll see what our data frame looks like actually to do that we need to go back up here and run all of this awesome go to our working directory open this up and they've all been formatted with this string format so that is good what we need to do now is we need to go up here and actually change the format that's being applied to each one so we have string template dollar template integer template float template a percent template and this float template here actually did run this should be zero run that and then we'll go through here and change each format according to the dollar this should be an integer this should be a float PE percentile should be a percent price-to-book ratio should be a float price-to-book ratio percentile should be a percent price-to-sales ratio should be a float price-to-sales ratio percentile should be a percent if you think that should be a float you need to even up percentile should be a percent you need to GP should be a float you need to GP percentile should be a percent and robust value score should also be a percent because it is an average of all the different percentiles in our pandas data frame so we'll run all this again and see what this looks like awesome so we have ticker which is a string price which is a dollar number of shares to buy which is an integer price-to-earnings ratio which is a flow PE percentile everything looks to be

good here great so the only other thing that needs change about this is that we need to format our columns and we also need to change our percentiles because just like in the momentum project they are two orders of magnitude bigger than what they should be so we'll fix the percentile score calculations first that's pretty easy all we need to do is scroll up to here find the loop where we calculated our percentile scores calculated by percentiles so if we divide this by 100 and that should fix the problem so we can do this run a bunch of code cells do a two and a half million dollar portfolio again and then we're back down to format our excel I'm actually gonna go through all of this and say that again just to make sure that that was done properly if we open this up and we go here we can now see that the percentiles all range from 0 to 100 as desired awesome the next thing we need to do is actually format our column headers so to do that we will say brighter dot sheets and then passing value strategy select these sheets that we want to format and then we call the right method now you'll recall that the right method takes a few different things it takes the cell we want to write so this will be for each row or sorry for each column it will be the the first cell of that column so we will say column and then one so that's the first cell of that column the second thing we need is the actual value that we want to write into that cell so that will be the first entry of each of these lists just like before so we paste that in and use index zero and then the last thing that it takes is the format that you want to apply to that cell so that just like the set column method that we use here is going to be the second entry of that list so we will pass in index one all right so that should be all the formatting what I'm going to do next is just move this writer dot save down to here I'm going to delete all of these empty cells and then I'm going to scroll back up to where we created our Excel file I'm going to run all the code and you should be good to go so this started here so I'm going to click that and then do shift enter until I reach the end of the Jupiter notebook and here's our final spreadsheet so as you can see all the formats have been applied successfully and you have now created a quantitative value investing strategy that identifies the 50 cheapest stocks in the S&P 500 using a basket of different valuation metrics and that concludes our third project so kudos to you for making it all the way through this algorithmic trading in Python course